

The Opus Audio Codec: Theory and Implementation

Jack Geraghty

Contents

1	Introduction	5
1.1	The Reference Implementation	5
1.2	Target Audience and Prerequisites	5
1.3	Scope and Organization	6
1.4	Notation Conventions	6
2	Architecture Overview	6
2.1	Two Codecs, One Bitstream	7
2.2	The Layered Stack	7
2.3	Decoding a Packet: An Overview	8
2.4	Sample Rates and the 48 kHz Reference	8
2.5	From Quantities to Symbols	9
3	The Range Coder	9
3.1	Why Entropy Coding	9
3.2	The Integer Range Coder	10
3.2.1	Decoding a symbol	10
3.2.2	The renormalization loop	11
3.2.3	Encoding a symbol	11
3.3	Carry Propagation	12
3.4	The ICDF Representation	12
3.5	Binary Symbols and Raw Bits	13
3.6	Bit Budgeting: tell() and tell_frac()	14
3.7	The Final-Range Oracle	15
4	Packet Framing	15
4.1	The Self-Delimiting Question	16
4.2	The TOC Byte	16
4.2.1	The 32 configurations	16
4.3	Frame-Packing Codes	17
4.3.1	Code 3 in detail	18

4.4	Validity Rules	18
5	CELT: Transform Coding	19
5.1	The MDCT and Lapped Transforms	19
5.1.1	The window and perfect reconstruction	20
5.1.2	Short blocks for transients	20
5.1.3	Computing the MDCT through an FFT	20
5.2	The Energy Envelope	21
5.2.1	Coarse energy and two-dimensional prediction	21
5.3	Pyramid Vector Quantization	22
5.3.1	The pulse search	22
5.4	CWRS: Enumerating the Codebook	23
5.5	Bit Allocation	23
5.6	Band Coding: Splits, Folding, and Stereo	24
5.6.1	Recursive splitting by angle	25
5.6.2	Spreading rotation	25
5.6.3	Folding and noise fill	25
5.6.4	Collapse masks and anti-collapse	25
5.6.5	Stereo coupling	25
5.6.6	Time-frequency reshaping	26
5.7	The Encoder's Analysis	26
6	SILK: Linear-Predictive Coding	27
6.1	The Source-Filter Model	27
6.2	Short-Term Prediction	27
6.2.1	Estimating the coefficients	28
6.3	Line Spectral Frequencies	29
6.3.1	The two-stage vector quantizer	30
6.3.2	Reconstruction and stability enforcement	30
6.4	Long-Term Prediction	30
6.5	Subframe Gains	31
6.6	The Excitation and the Shell Coder	31
6.7	The Noise-Shaping Quantizer	32
6.7.1	Noise-shaping analysis	33
6.8	Stereo Prediction	33
6.9	Resampling	34
6.10	Voice Activity, Pitch, and Rate Control	34
6.11	Packet-Loss Concealment in SILK	34
7	The Opus Layer	34

7.1	Mode Dispatch	34
7.2	Hybrid Mode and the Crossover	35
7.3	Mode Transitions and Redundancy	35
7.4	Packet-Loss Concealment	36
7.5	In-Band Forward Error Correction	36
7.6	Discontinuous Transmission	37
7.7	The Encoder’s Mode Selection	37
8	Ogg Encapsulation	37
8.1	Pages and Lacing	37
8.2	The Ogg CRC	38
8.3	Packet Reassembly	38
8.4	OpusHead and OpusTags	39
8.5	Granule Positions and Timing	39
9	Multistream and Surround	39
9.1	Streams and Coupling	40
9.2	Demultiplexing	40
10	Computational Optimizations	40
10.1	The Licence to Approximate	41
10.2	SIMD Dot Products	41
10.3	The SIMD PVQ Search	42
10.4	The MDCT Pre-Rotation	42
10.5	The FFT Backend Seam	42
10.6	Table-Driven Enumeration and Buffer Reuse	42
11	Numerical Considerations and Conformance	43
11.1	Fixed-Point and Floating-Point	43
11.2	What Must Be Exact	43
11.3	The Conformance Criterion	43
12	Python API Reference	44
12.1	Introduction	44
12.2	Installation	44
12.3	Quickstart: Encode and Decode	45
12.4	Choosing a Codec Mode	46
12.5	Frame Sizes	46
12.6	Ogg Opus Files	47
12.7	Packet Loss and Forward Error Correction	48
12.8	Multistream and Surround Decoding	48

12.9 Packet Introspection	49
12.10The Low-Level API	49
12.11Complete API Reference	50

1 Introduction

This manual provides a comprehensive overview of the algorithms, optimizations, and design decisions that make up the Opus audio codec [16]. The focus is on understanding both the theoretical foundations and the practical implementations of the techniques Opus combines: the information theory of its entropy coder, the linear prediction of its speech path, the transform coding of its music path, and the fixed-point and SIMD machinery that lets all of it run in real time.

The material emphasizes the *why* and *how* of each component: *why* certain design choices matter, and *how* they are implemented efficiently. It covers both the mathematics and the practical computational considerations that make these methods work in production systems.

Opus is the IETF’s general-purpose, royalty-free audio codec, standardized in 2012 and updated by RFC 8251 [15]. It is the default audio codec of WebRTC and is widely deployed in streaming and conferencing. Its defining characteristic is that it is not one codec but two, combined behind a single bitstream. A linear-predictive speech coder (***SILK***) and a transform-domain music coder (***CELT***, the Constrained-Energy Lapped Transform) share one entropy coder, one packet format, and one container. The encoder selects, per frame, between SILK, CELT, or both together (the ***hybrid*** mode). The combination spans a range no single technique covers well: 6 kb/s narrowband speech to 510 kb/s fullband stereo music, frame sizes from 2.5 ms to 60 ms, and sample rates from 8 kHz to 48 kHz, all decoded by one receiver that is not told in advance which mode each packet uses.

1.1 The Reference Implementation

Throughout, the implementation under discussion is **ruopus**, a pure-Rust realization of RFC 6716 with zero *required* runtime dependencies (the default build pulls in one optional FFT crate for faster decoding; `default-features = false` is fully dependency-free). Every code listing in this manual is drawn from that source. Three of its design choices shape the discussion:

- **Bit-exact by construction.** The bitstream is defined by the range coder (Section 3), not by any one encoder’s choices. The implementation’s entropy arithmetic follows the RFC byte-for-byte, and the decoder is validated against the official test vectors on the *final-range oracle*, a per-packet checksum of the entropy-coder state that must agree with the reference to the bit.
- **Safe by default.** The crate denies `unsafe` everywhere except a handful of explicitly annotated SIMD kernels (Section 10). The signal-processing math is ordinary safe Rust; the only machine intrinsics sit in encoder hot loops where a faster, approximate result is provably harmless (Section 10).
- **Decode-first.** The decoder is the normative half of the specification and the half the conformance vectors exercise. The encoder is free to make any choice the decoder can follow; this asymmetry recurs throughout the manual and is the key to several optimizations.

1.2 Target Audience and Prerequisites

This material assumes familiarity with:

- Discrete signals, sampling, and the discrete Fourier transform
- Linear algebra (vectors, matrices, inner products, projection)
- Basic probability and the idea of entropy / information content
- Programming and algorithm analysis; the listings are in Rust, but no Rust expertise is required to follow them

No prior knowledge of speech coding, transform coding, arithmetic coding, or psychoacoustics is assumed; each is introduced where it is first needed. A reader who has worked through a companion

treatment of FFT-based spectral analysis will find the transform-coding sections (CELT) immediately familiar, and the linear-prediction sections (SILK) a complementary story told in the time domain.

1.3 Scope and Organization

The manual is organized bottom-up, from the bit to the file, mirroring the layering of the codec itself:

1. **Architecture Overview** (Section 2) — the two codecs, three modes, the shared range coder, and the sample-rate model.
2. **The Range Coder** (Section 3) — entropy coding, arithmetic/range coding, the ICDF representation, raw bits, `tell()`, and the final-range oracle.
3. **Packet Framing** (Section 4) — the TOC byte, the 32 configurations, frame packing codes 0–3, padding, and validity rules.
4. **CELT: Transform Coding** (Section 5) — the MDCT, the Vorbis window and TDAC, pyramid vector quantization, CWRS enumeration, band splitting, energy coding, bit allocation, and the encoder’s analysis.
5. **SILK: Linear-Predictive Coding** (Section 6) — linear prediction, LSF/NLSF quantization, long-term (pitch) prediction, the noise-shaping quantizer, gains, the shell coder, and resampling.
6. **The Opus Layer** (Section 7) — mode dispatch, the SILK/CELT crossover, redundancy at transitions, packet-loss concealment, in-band FEC, and discontinuous transmission.
7. **Ogg Encapsulation** (Section 8) — pages, the Ogg CRC, lacing, packet reassembly, `OpusHead/OpusTags`, and granule timing.
8. **Multistream and Surround** (Section 9) — coupled and uncoupled streams, channel mapping families.
9. **Computational Optimizations** (Section 10) — SIMD dot products, the SIMD PVQ search, the MDCT pre-rotation, the FFT backend seam, table-driven CWRS, and buffer reuse.
10. **Numerical Considerations and Conformance** (Section 11) — fixed-point versus float, bit-exactness, the oracle, and the conformance criterion.

1.4 Notation Conventions

Throughout this manual:

- $x[n]$ denotes a time-domain sample at index n ; $X[k]$ a frequency-domain coefficient at bin k
- f_s denotes a sample rate in Hz; Opus’s internal reference rate is always 48 kHz
- N denotes a transform length or vector dimension; K a pulse count
- LM (“log multiplier”) denotes $\log_2$ of the frame-size multiplier: $LM = 0, 1, 2, 3$ for 2.5, 5, 10, 20 ms CELT frames
- The range coder’s state is the pair (val, rng)
- Q_n denotes a fixed-point value scaled by 2^n : a Q15 value v represents the real number $v/2^{15}$
- $\text{sgn}(x)$ is the sign of x ; $\lfloor \cdot \rfloor$ and $\lceil \cdot \rceil$ are floor and ceiling
- Section references like “§4.3.4” without further qualification refer to RFC 6716 [16]

2 Architecture Overview

This section describes the codec as a whole: its components, the path a packet takes through them, and the design decisions that the rest of the manual elaborates. Later sections treat each layer in detail.

2.1 Two Codecs, One Bitstream

Opus is built on the observation that audio divides into two regimes that reward opposite coding strategies. The codec implements both and selects between them per frame.

Speech versus Music: Two Coding Strategies

Speech is produced by a physical system (the vocal folds exciting the vocal tract) that is well modeled as a slowly varying *linear filter* driven by a sparse *excitation*. Coding speech efficiently means coding that model: estimate the filter, estimate the pitch, and transmit only the residual that the model fails to predict. This is **linear-predictive coding** (LPC), and it is the approach SILK takes, in the *time domain*, at internal rates of 8, 12, or 16 kHz.

Music is arbitrary: dense, polyphonic, and percussive, with energy spread across the full 20 kHz audible band and no simple source model. Coding music efficiently means working in the *frequency domain*, where the ear’s own frequency analysis allows bits to be spent where they are audible and withheld where masking hides them. This is **transform coding**, and it is the approach CELT takes, via the Modified Discrete Cosine Transform (MDCT) at 48 kHz.

Neither technique performs both tasks well: LPC degrades on polyphonic material, and transform coding spends bits inefficiently on the strong predictable structure of speech. Opus therefore retains both and switches between them per frame; in the overlap region (wideband speech with a musical background) it runs them together as the **hybrid** mode.

This yields the three **modes**, selected by the packet header (Section 4) and dispatched by the decoder (Section 7):

Mode	Coder(s)	Typical use
SILK-only	SILK	low-bitrate speech, 8–16 kHz audio band
Hybrid	SILK (low) + CELT (high)	wideband speech + full bandwidth
CELT-only	CELT	music, low delay, high bitrate

In hybrid mode the split is by frequency: SILK codes the signal up to roughly 8 kHz (it runs at a 16 kHz internal rate), and CELT codes the band above, starting at MDCT band 17 (Section 7.2). Both coders write into the same range coder, so there is no boundary marker between them in the byte stream: the CELT data is simply a continuation of the same range-coded message that began with the SILK data.

2.2 The Layered Stack

Table 1 lists the layers as the implementation modularizes them, bottom to top. The lower layers (range coder, packet, Ogg) are pure framing and entropy: exact integer arithmetic, `no_std`-compatible, with no audio math. The middle layers (SILK, CELT) perform the signal processing. The top layer (the Opus decoder and encoder) provides the glue: mode dispatch, resampling, and the cross-frame state used to handle mode transitions.

Layer	Specification	Responsibility
ogg	RFC 3533 / 7845	file container, pages, timing
multistream	RFC 7845 §5.1.1	surround: N coupled/mono streams
decoder/encoder	RFC 6716 §4/§5	mode dispatch, transitions, PLC/FEC/DTX
silk	RFC 6716 §4.2	linear-predictive speech coder
celt	RFC 6716 §4.3	MDCT transform coder
packet	RFC 6716 §3	TOC byte, frame packing
range	RFC 6716 §4.1/§5.1	entropy coder

Table 1: The Opus layer stack, bottom to top.

2.3 Decoding a Packet: An Overview

To decode one Opus packet, the decoder performs the following steps:

1. Reads the first byte, the **TOC** (Table of Contents), which names the mode, audio bandwidth, frame size, and channel count (Section 4).
2. Splits the remaining bytes into 1–48 *frames* according to the TOC’s frame-packing code.
3. For each frame, constructs a range decoder over its bytes and, depending on the mode, runs the SILK decoder, the CELT decoder, or both into a shared range decoder.
4. Resamples the decoder’s internal output (always referenced to 48 kHz) to the caller’s requested rate, and applies any cross-frame smoothing for mode transitions and redundancy (Section 7).

The entry point parses the packet and hands it to a per-frame driver, itself a long chain of mode- and bandwidth-conditioned branches rather than one tidy dispatch, since transitions and hybrid redundancy each need their own handling:

```
pub fn decode_packet(&mut self, data: &[u8]) -> Result<Vec<f32>,
PacketError> {
    // Empty or TOC-only packet: conceal one frame (DTX / loss).
    if data.len() <= 1 {
        return Ok(self.decode_lost(self.last_frame_size / self.ds));
    }
    let packet = Packet::parse(data)?;
    Ok(self.decode_parsed(&packet))
}

fn decode_parsed(&mut self, packet: &Packet) -> Vec<f32> {
    let toc = packet.toc();
    self.stream_channels = usize::from(toc.channels());
    let mut out = Vec::new();
    for frame in packet.frames() {
        let pcm = self.decode_frame(
            frame, toc.mode(), toc.bandwidth(),
            toc.frame_size().samples_per_channel_48k(),
        );
        out.extend_from_slice(&pcm);
    }
    out
}
```

2.4 Sample Rates and the 48 kHz Reference

The relationship between sample rates in Opus is a common source of confusion. The model is stated once here and referred to throughout.

The 48 kHz Reference Clock

Opus has three distinct notions of sample rate:

- The **audio bandwidth** (narrowband, mediumband, wideband, super-wideband, fullband) is a *property of the content*: how much of the spectrum carries signal. It is named in the TOC and caps the highest coded frequency (4, 6, 8, 12, or 20 kHz).
- The **internal rate** is the rate a coder actually runs at. SILK runs at 8, 12, or 16 kHz; CELT always runs at 48 kHz.
- The **API rate** is the rate the caller asks for (8, 12, 16, 24, or 48 kHz), at which PCM is delivered.

Crucially, **all timing is expressed in 48 kHz samples** regardless of the other two. A 20 ms frame is always 960 samples “at 48 kHz”; a 2.5 ms frame is 120. Granule positions in the Ogg container (Section 8) count 48 kHz samples. The decoder resamples from the internal rate up to 48 kHz (CELT is already there), then *down* to the API rate by an integer factor $ds = 48000/f_s$. This single reference clock is what lets SILK frames, CELT frames, and the container agree on “when” a sample occurs even though they compute it at different rates.

2.5 From Quantities to Symbols

Every numeric quantity Opus transmits (a gain, an LSF index, a pulse vector, an energy delta) becomes a symbol in the range coder. The encoder and decoder share static probability models (**ICDF** tables, Section 3) for the structured fields and fall back to **raw bits** for the near-uniform ones. This arrangement has an important consequence:

Important Note

The bitstream is defined by the *sequence of range-coder symbols*, not by the floating-point intermediate values the encoder computed to choose them. Two encoders that pick the same symbols produce identical bytes, even if one used SIMD and `rsqrt` approximations and the other used scalar double-precision. This is why the decoder must be bit-exact (it is normative) but the encoder need only be *valid* (any choice the decoder can follow is legal). Most of Section 10 exploits exactly this freedom.

3 The Range Coder

Every bit Opus transmits passes through the range coder. Because the bitstream is defined entirely by the sequence of range-coder symbols, and because bit-exact conformance is verified through the coder’s internal state (Section 3.7), it is the natural starting point and is treated here first and in detail.

3.1 Why Entropy Coding

Suppose a symbol s occurs with probability $p(s)$. Information theory says the *ideal* cost of transmitting it is its **self-information**, $-\log_2 p(s)$ bits [13, 2]. A common symbol (p near 1) should cost a fraction of a bit; a rare one, many bits. Over a long message drawn from a known distribution, the average cost approaches the **entropy** $H = -\sum_s p(s) \log_2 p(s)$, and no lossless code can do better.

A fixed-length code cannot reach this bound: it must spend a whole number of bits per symbol, wasting the fractional part. **Arithmetic coding**, and its integer-friendly cousin **range coding** [8, 17], do reach it, by abandoning the idea that each symbol maps to its own bit pattern. Instead, the *entire message* is encoded as a single number in $[0, 1)$, and each symbol narrows the interval that number must lie in.

Range Coding by Interval Subdivision

Begin with the interval $[0, 1)$. To encode a symbol s whose cumulative distribution carves out the sub-range $[fl/ft, fh/ft)$ (low and high cumulative frequencies out of a total ft), *replace* the current interval with that fraction of itself:

$$[\ell, \ell + r) \mapsto \left[\ell + r \frac{fl}{ft}, \ell + r \frac{fh}{ft} \right). \quad (1)$$

The interval shrinks by a factor of $(fh - fl)/ft = p(s)$ at each step, so after the whole message its width is $\prod_i p(s_i) = 2^{-\sum_i -\log_2 p(s_i)}$. Transmitting *any* number inside the final interval takes about $-\log_2(\text{width}) = \sum_i -\log_2 p(s_i)$ bits, the entropy. The decoder, knowing the same model, replays the subdivisions: it sees which sub-range the received number falls in, emits that symbol, rescales, and repeats.

The practical obstacle is that the interval bounds need ever more precision as they shrink, which is impossible with finite-width integers. The fix, due to Pasco and refined by Martin, is **renormalization**: as soon as the interval is narrow enough that its top bits are settled, *emit those bits* and shift them out, restoring headroom. Opus renormalizes a byte at a time.

3.2 The Integer Range Coder

Opus’s range coder (§4.1 for the decoder, §5.1 for the encoder) works in 32-bit integers. Its state is two words:

- **rng**: the current interval width, kept in $(2^{23}, 2^{31}]$.
- **val**: on the encoder, the interval’s low end; on the decoder, the offset of the received number from the top of the interval.

The governing constants name the renormalization threshold and the byte width:

```
const SYM_BITS:  u32 = 8;           // emit/consume one byte at a time
const SYM_MAX:    u32 = 255;         // 2^8 - 1
const CODE_BITS:  u32 = 32;          // width of the (val, rng) state
const CODE_TOP:   u32 = 1 << 31;     // 2^31: initial encoder range
const CODE_BOT:   u32 = 1 << 23;     // 2^23: renormalize when rng
// falls to here
const CODE_SHIFT: u32 = 23;          // CODE_BITS - SYM_BITS - 1; carry
// bit position

#[inline]
const fn ilog(x: u32) -> u32 { x.leading_zeros() } // 1 + floor(log2 x)
```

The lower bound $rng > 2^{23}$ guarantees that the division rng/ft in Equation (1) keeps enough significant bits for the largest total frequency Opus uses ($ft \leq 2^{16}$). Note `ilog` is just the hardware count-leading-zeros instruction; it appears everywhere the coder needs $\lfloor \log_2 \rfloor$.

3.2.1 Decoding a symbol

Decoding is two steps: **decode** reports *where* the received value sits in $[0, ft)$, the caller maps that to a symbol, then **update** shrinks the state to that symbol’s sub-range and renormalizes.

```

pub fn decode(&mut self, ft: u32) -> u32 {
    // Locate val within [0, ft): the symbol s satisfies fl <= decode()
    < fh.
    ft - (self.val / (self.rng / ft) + 1).min(ft)
}

pub fn update(&mut self, fl: u32, fh: u32, ft: u32) {
    let s = self.rng / ft;
    self.val -= s * (ft - fh);           // shift past the symbols
    above this one
    self.rng = if fl > 0 { s * (fh - fl) } // interior symbol: exact
    sub-width
    else { self.rng - s * (ft - fh) }; // first symbol
    keeps remainder
    self.normalize();
}

```

Two details are easy to miss and important to get right. First, the first symbol ($fl = 0$) is given the *remainder* of the range ($rng - s(ft - fh)$), not $s \cdot fh$; this is how the coder accounts for the truncation in $s = \lfloor rng/ft \rfloor$ and stays reversible. Second, `decode` caps its result at ft via `.min(ft)` to defend against a corrupt stream pushing the quotient out of range.

3.2.2 The renormalization loop

When `rng` drops to 2^{23} or below, the decoder shifts in a fresh byte:

```

fn normalize(&mut self) {
    while self.rng <= CODE_BOT {
        self.nbits_total += SYM_BITS;
        self.rng <= SYM_BITS;
        // Reassemble the next 8-bit symbol from the leftover bit of
        the
        // previous byte (high) and the top 7 bits of the new byte (low
        ).
        let byte = self.read_byte();
        let sym = (self.leftover_bit << 7) | (byte >> 1);
        self.leftover_bit = byte & 1;
        self.val = ((self.val << SYM_BITS) + (SYM_MAX - sym)) & 0
        x7FFF_FFFF;
    }
}

```

The one-bit shimmy (`leftover_bit`) is a peculiarity of Opus’s coder: the decoder works with a 7-bits-from-this-byte, 1-bit-carried representation so that the encoder’s carry propagation (below) lands on byte boundaries. The `& 0x7FFF_FFFF` keeps `val` within 31 bits.

3.2.3 Encoding a symbol

The encoder mirrors the decoder exactly; it must, or the two will disagree and the stream will be undecodable:

```
pub fn encode(&mut self, fl: u32, fh: u32, ft: u32) {
    let r = self.rng / ft;
    if fl > 0 {
        self.val += self.rng - r * (ft - fl);
        self.rng = r * (fh - fl);
    } else {
        self.rng -= r * (ft - fh);
    }
    self.normalize();
}
```

The encoder's `normalize` emits the top byte of `val` instead of reading one, but is otherwise the same loop, with one complication that deserves its own treatment.

3.3 Carry Propagation

When the encoder adds to `val` (the `fl > 0` branch above), the addition can overflow into bits it has *already emitted*. A byte written out as `0xFF` might need to become `0x00` with a carry rippling into the byte before it. The encoder cannot un-write bytes, so it does not write them eagerly. Instead it buffers:

```
fn carry_out(&mut self, c: u32) {
    if c == SYM_MAX {
        self.ext += 1; // a run of 0xFF: defer, might
        all carry
    } else {
        let carry = (c >> SYM_BITS) as u8;
        if let Some(rem) = self.rem.take() {
            self.write_byte(rem + carry); // resolve the buffered
            byte
        }
        if self.ext > 0 {
            // Flush the deferred 0xFF run, +carry turns them into 0x00
            .
            let sym = (SYM_MAX + u32::from(carry)) as u8;
            for _ in 0..self.ext { self.write_byte(sym); }
            self.ext = 0;
        }
        self.rem = Some((c & SYM_MAX) as u8);
    }
}
```

Why a 0xFF Run Must Be Held

A carry into a `0xFF` byte turns it into `0x00` and *propagates further left*. A carry into any other byte simply increments it and stops. So the encoder can safely emit a non-`0xFF` byte only once it knows whether a carry is coming, which it learns from the *next* byte. It therefore keeps one pending byte in `rem` and counts a run of `0xFF`s in `ext`. When a non-`0xFF` byte arrives with carry `c`: the pending byte becomes `rem + c`, and the held `0xFF`s become either `0xFF` (no carry) or `0x00` (carry). This buffering is invisible to the decoder; it simply reads the bytes that finally emerge. The mechanism is what makes the encoder/decoder *symmetric to the bit*, which is the property the conformance oracle (Section 3.7) checks.

3.4 The ICDF Representation

Structured fields are coded against static probability tables. Opus stores them not as a PDF or a forward CDF but as an *inverse cumulative distribution function* (ICDF): for a model with total

2^{ftb} , the table holds $icdf[k] = 2^{ftb} - fh[k]$, the cumulative *tail* above each symbol, decreasing to a terminating zero.

Why Inverse CDFs

Two reasons. First, the decode primitive in Equation (1) naturally produces “how far down from the top is *val*”, which is exactly an upper-tail comparison, so an ICDF lets the decoder walk the table with one multiply and one compare per step and no running subtraction. Second, the tables are tiny (a handful of bytes), and storing the tail lets the common “most probable symbol first” ordering terminate early. The forward arithmetic (fl, fh) is recovered implicitly from adjacent ICDF entries.

```
pub fn decode_icdf(&mut self, icdf: &[u8], ftb: u32) -> usize {
    let d = self.val;
    let r = self.rng >> ftb;           // ft is a power of two: shift,
        don't divide
    let mut k = 0;
    let mut t = self.rng;
    let mut s = r * u32::from(icdf[0]);
    while d < s {                       // walk down the tail until val
        sits in [s, t)
        k += 1;
        t = s;
        s = r * u32::from(icdf[k]);
    }
    self.val = d - s;                   // fold update() into the same
        step
    self.rng = t - s;
    self.normalize();
    k
}
```

Because $ft = 2^{ftb}$ is a power of two, the division rng/ft becomes a shift, a small but pervasive saving, since ICDF decoding is the hottest path in SILK and in CELT’s side information.

3.5 Binary Symbols and Raw Bits

Two specializations round out the coder.

Binary symbols with a power-of-two probability. Many flags have a probability of the form 2^{-logp} . These skip the table entirely:

```
pub fn decode_bit_logp(&mut self, logp: u32) -> bool {
    let s = self.rng >> logp;         // sub-range for the "1" outcome
    let bit = self.val < s;
    if !bit { self.val -= s; }
    self.rng = if bit { s } else { self.rng - s };
    self.normalize();
    bit
}
```

Raw bits. Some quantities are effectively uniform (a sign, the low bits of a pulse split), and modeling them as probabilities wastes effort. Opus codes these as *raw bits*, written at the *opposite end* of the same buffer from the arithmetic-coded data.

The Two-Ended Buffer

A range-coded packet has *two* write heads in one byte array. The arithmetic coder fills from the front, most-significant first, byte by byte as it renormalizes. Raw bits fill from the *back*, least-significant first. The two heads grow toward each other; the packet is well-formed as long as they do not cross. This costs nothing: raw bits need no probability model, so they need not interleave with the arithmetic stream, and it lets the finalizer (Section 3.6) pack the last arithmetic byte and the last raw byte into the same physical byte when they happen to fit. The decoder reads raw bits backwards from the end of the packet through a small 32-bit window, refilling a byte at a time.

```
pub fn decode_raw_bits(&mut self, bits: u32) -> u32 {
    if self.nend_bits < bits {                                // refill the
        window from the end
        loop {
            self.end_window |= self.read_byte_from_end() << self.
                nend_bits;
            self.nend_bits += SYM_BITS;
            if self.nend_bits > WINDOW_SIZE - SYM_BITS { break; }
        }
    }
    let ret = self.end_window & ((1 << bits) - 1);          // LSB-first
    self.end_window >>= bits;
    self.nend_bits -= bits;
    self.nbits_total += bits;                                // tell()/
    tell_frac() count these too
    ret
}
```

Uniform integers combine the two: a value in $[0, ft)$ with ft not a power of two is split into an arithmetic-coded high part (8 bits of model) and a raw-bit low part.

```
pub fn encode_uint(&mut self, t: u32, ft: u32) {
    let ftb = ilog(ft - 1);
    if ftb <= 8 {
        self.encode(t, t + 1, ft);                            // small: pure
        arithmetic
    } else {
        let ft_hi = ((ft - 1) >> (ftb - 8)) + 1;
        self.encode(t >> (ftb - 8), (t >> (ftb - 8)) + 1, ft_hi);
        self.encode_raw_bits(t & ((1 << (ftb - 8)) - 1), ftb - 8); //
        low bits raw
    }
}
```

3.6 Bit Budgeting: tell() and tell_frac()

Opus's bit allocation (Section 5.5) needs to know, mid-frame, exactly how many bits have been spent so far, and it needs that figure to *agree* between encoder and decoder, since both run the same allocation arithmetic from the same running total. The coder exposes this through `tell` and its eighth-of-a-bit refinement `tell_frac`.

```

pub fn tell(&self) -> u32 { self.nbits_total - ilog(self.rng) }

pub(crate) fn tell_frac(nbits_total: u32, rng: u32) -> u32 {
    let mut lg = ilog(rng);
    let mut r = rng >> (lg - 16);    // Q15 fractional part of log2(
    rng)
    for _ in 0..3 {                  // 3 iterations -> 1/8-bit
        resolution
        r = (r * r) >> 15;
        let bit = r >> 16;
        lg = 2 * lg + bit;
        r >>= bit;
    }
    nbits_total * 8 - lg
}

```

Bits Used: Total Shifted In Minus Range Headroom

The intuition: `nbits_total` counts every bit shifted into the coder. But the coder has not *committed* all of them: the current range `rng` still spans $\log_2(rng)$ bits of undecided value. So the bits actually consumed are `nbits_total` $- \log_2(rng)$. `tell()` uses the integer $\lfloor \log_2 \rfloor$ (a conservative whole-bit count); `tell_frac()` refines $\log_2(rng)$ to 3 fractional bits by the Newton-style squaring loop above, giving the 1/8-bit resolution Opus’s allocator works in. The two readouts agree on encoder and decoder because they are computed from the same *(nbits_total, rng)*.

3.7 The Final-Range Oracle

The range coder provides a single, powerful correctness check for any Opus implementation, described below.

rng-Agreement and the Final Range

After encoding (or decoding) the same sequence of symbols, the encoder and decoder must hold *identical* `rng` values, and, transitively, identical `val` and identical output bytes. RFC 6716 elevates this to a conformance tool: `OPUS_GET_FINAL_RANGE` returns the coder’s `rng` at the end of each packet, and a correct decoder’s final range must equal the reference encoder’s, *exactly*, for every packet.

This is a strong check. The entire decode (thousands of symbols across SILK and CELT, every table lookup, every renormalization) collapses into a single 32-bit number that is wrong if *any* step diverged. The `ruopus` conformance suite enforces it on all twelve official vectors: a passing decode is not merely close to the reference, it is arithmetically identical through the entropy coder. The PCM may differ by a rounding bit in the final float MDCT; the *range* may not differ at all.

In hybrid and redundancy cases (Section 7) the packet carries more than one range-coded segment; the implementation XORs their final ranges together to form the single reported value, matching the reference’s bookkeeping.

4 Packet Framing

The range coder produces the bytes of a single frame. The packet layer (§3) is the envelope around those bytes: it names how to interpret the frame, allows several frames to share one packet, and defines the rules a decoder uses to reject malformed input. It contains no audio math and no entropy coding, only parsing.

4.1 The Self-Delimiting Question

A design decision shapes the entire packet format: an Opus packet is *not* self-delimiting. It does not record its own length. Opus assumes a transport layer (Ogg, RTP, WebRTC, Matroska) that already knows where each packet ends, and declines to duplicate that information.

Why Packets Carry No Length Field

Storing a length in every packet would waste two or three bytes on data the transport already has. At the low bitrates Opus targets, a 20 ms packet can be under 20 bytes, so a redundant length prefix is a 10–15% overhead. Opus instead infers frame boundaries from the total packet size supplied by the container. The cost is that the codec cannot be handed a bare byte stream and asked to find packet boundaries; it must be told, per packet, how many bytes belong to it. The one exception is the *self-delimiting* framing used inside multistream packets (Section 9), where several Opus packets are concatenated and all but the last must announce their own length.

4.2 The TOC Byte

Every packet begins with one *Table of Contents* (TOC) byte. It packs three fields that, together, determine everything the decoder needs to dispatch the packet:

$$\underbrace{\text{C C C C C}}_{\text{config (5)}} \quad \underbrace{\text{S}}_{\text{stereo (1)}} \quad \underbrace{\text{n n}}_{\text{count code (2)}} \quad (2)$$

The top five bits select one of 32 *configurations*; the next bit is the mono/stereo flag; the low two bits are the frame-packing code (Section 4.3). The implementation treats the TOC as a transparent wrapper over the byte, with each field a small accessor:

```
pub struct Toc(u8);

impl Toc {
    pub const fn config(self) -> u8 { self.0 >> 3 } // bits 7..3
    pub const fn stereo(self) -> bool { (self.0 >> 2) & 1 == 1 } // bit 2
    pub const fn channels(self) -> u8 { 1 + ((self.0 >> 2) & 1) }
    pub const fn frame_count_code(self) -> u8 { self.0 & 0x3 } // bits 1..0
    pub const fn mode(self) -> Mode { /* from config */ }
    pub const fn bandwidth(self) -> Bandwidth { /* from config */ }
    pub const fn frame_size(self) -> FrameSize { /* from config */ }
}
```

4.2.1 The 32 configurations

The configuration number is not an arbitrary index; it is a structured encoding of (mode, bandwidth, frame size). The 32 values partition into the three modes, and within each mode the bandwidths and frame sizes are laid out in contiguous runs (Table 2). This regularity lets the encoder compute a config as a base value plus a frame-size offset, which is exactly how the encoder builds its TOC (Section 7).

Three properties of Table 2 are worth noting. SILK-only modes support the long 40 and 60 ms frames that suit speech but never the short 2.5 and 5 ms frames; CELT-only modes are the reverse, supporting short low-delay frames down to 2.5 ms. Hybrid mode exists only at the two highest bandwidths, because it is defined as “SILK wideband plus a CELT extension”; below super-wideband there is no high band for CELT to add. And mediumband appears only for SILK, as CELT’s band layout has no 6 kHz cutoff.

Config	Mode	Bandwidth	Frame sizes (ms)
0–3	SILK-only	narrowband (NB)	10, 20, 40, 60
4–7	SILK-only	mediumband (MB)	10, 20, 40, 60
8–11	SILK-only	wideband (WB)	10, 20, 40, 60
12–13	Hybrid	super-wideband (SWB)	10, 20
14–15	Hybrid	fullband (FB)	10, 20
16–19	CELT-only	narrowband (NB)	2.5, 5, 10, 20
20–23	CELT-only	wideband (WB)	2.5, 5, 10, 20
24–27	CELT-only	super-wideband (SWB)	2.5, 5, 10, 20
28–31	CELT-only	fullband (FB)	2.5, 5, 10, 20

Table 2: The 32 TOC configurations (§3.1, Table 2). The mode dictates which coder runs; the bandwidth caps the highest coded frequency; the frame size is always expressed at the 48 kHz reference.

4.3 Frame-Packing Codes

The two low bits of the TOC select one of four ways to pack frames into the remaining bytes. The motivation is amortization: a single packet can carry up to 120 ms of audio (for example three 40 ms frames), spreading the one-byte TOC overhead across multiple frames and reducing per-frame container overhead.

The Four Frame-Packing Codes

Let the bytes after the TOC be the *payload*. The count code determines how the payload splits into frames:

- **Code 0:** one frame, occupying the entire payload.
- **Code 1:** two frames of *equal* size; the payload length must be even, and each frame is half.
- **Code 2:** two frames of possibly *unequal* size; an explicit length precedes the first frame, and the second takes the rest.
- **Code 3:** an arbitrary count M of frames (1–48), with a following byte that selects constant-bitrate (CBR, all frames equal) or variable-bitrate (VBR, explicit lengths) packing, and optional padding.

Frame lengths, where coded explicitly (code 2 and VBR code 3), use a one- or two-byte representation: a byte below 252 is the length itself; a byte of 252 to 255 is combined with a following byte as $\text{second} \times 4 + \text{first}$, reaching up to 1275, the maximum frame size. A coded length of zero is legal and denotes a dropped or DTX (discontinuous transmission) frame that the decoder conceals.

The parser is a direct transcription of these four cases:

```

match toc.frame_count_code() {
  0 => { check_frame_len(rest.len())?; frames.push(rest); }
  1 => {
    if rest.len() % 2 != 0 { return Err(PacketError::
      Code1UnevenPayload); }
    let (a, b) = rest.split_at(rest.len() / 2);
    frames.push(a); frames.push(b);
  }
  2 => {
    let n1 = read_frame_len(&mut rest)?; // explicit first
    length
    let frame1 = take(&mut rest, n1)?;
    frames.push(frame1); frames.push(rest); // second takes the
    remainder
  }
  _ => { /* code 3: count byte, optional padding, CBR or VBR lengths
    */ }
}

```

4.3.1 Code 3 in detail

Code 3 carries a second header byte after the TOC: bit 7 is the VBR flag, bit 6 signals the presence of padding, and the low six bits give the frame count M . Padding, when present, is a length encoded as a run of bytes (a value of 255 means “254 bytes of padding, continue reading”), allowing a packet to be inflated to a target size for constant-rate transport without affecting the audio. In VBR packing the header is followed by $M - 1$ explicit frame lengths, with the final frame taking the remainder; in CBR packing all M frames share the payload equally, and a payload not divisible by M is an error.

4.4 Validity Rules

RFC 6716 §3.4 enumerates seven conditions, labeled [R1] through [R7], that a well-formed packet must satisfy. They exist because a decoder fed hostile or corrupt input must reject it cleanly rather than read out of bounds or allocate unboundedly. The implementation’s error type documents which rule(s) each variant enforces (Table 3); several rules share an error because they are checked by the same code path, and R4/R6/R7 each contribute to more than one variant.

Error	Rule(s)	Condition
Empty	R1	packet is at least one byte (has a TOC)
FrameTooLarge	R2	every frame is at most 1275 bytes
Code1UnevenPayload	R3	code 1 payload length is even
InvalidFrameLength	R4, R7	a coded frame length is truncated or overruns the payload
InvalidFrameCount	R5	code 3 count $M \geq 1$ and total ≤ 120 ms
InvalidPadding	R6, R7	padding overruns the payload
CbrPayloadNotDivisible	R6	a CBR (code 2/3, non-VBR) payload does not divide evenly by the frame count

Table 3: The malformed-packet rules [R1]–[R7] (§3.4) and the typed errors that enforce them.

Important Note

Rules R5 and R6 are the codec’s defense against denial-of-service input. R5 caps a packet at 120 ms of audio, so a single packet cannot demand an unbounded decode; the implementation checks $M \times (\text{tenths of ms per frame}) \leq 1200$. R6 ensures the padding count and the CBR frame division, both attacker-controlled, cannot point past the end of the buffer or hide a malformed split. The packet rules are the first line that stops such input from reaching the signal-processing layers at all; the decoder is also exercised by differential fuzzing against the reference implementation (Section 11.3) to catch the arithmetic edge cases the rules alone do not.

5 CELT: Transform Coding

CELT, the Constrained-Energy Lapped Transform (§4.3), is the music half of Opus. It is a transform codec: it maps blocks of audio into the frequency domain with a Modified Discrete Cosine Transform, then spends bits on the resulting coefficients according to a psychoacoustic budget. Its distinguishing feature, the one its name advertises, is that it codes the *energy* of each frequency band explicitly and separately from the band’s *shape*, which keeps the reconstructed spectrum’s energy envelope faithful even when bits are scarce.

This section follows the decoder’s data flow: the transform (Section 5.1), the energy envelope (Section 5.2), the pyramid vector quantizer that codes band shapes (Section 5.3), the combinatorial enumeration that turns a quantized shape into an index (Section 5.4), the bit allocation that ties them together (Section 5.5), the band-coding machinery (Section 5.6), and finally the encoder’s analysis (Section 5.7).

5.1 The MDCT and Lapped Transforms

A transform codec cannot simply cut the signal into blocks and transform each independently: the discontinuities at block boundaries would produce audible clicks, and the rectangular truncation would smear energy across frequencies (spectral leakage). The standard remedy is to overlap consecutive blocks and window them, but naive overlap doubles the data rate. The Modified Discrete Cosine Transform [10, 11] resolves this tension: it overlaps blocks by 50% yet remains *critically sampled*, producing exactly N coefficients for every N new input samples.

The Modified Discrete Cosine Transform

The MDCT maps $2N$ real input samples to N real coefficients:

$$X[k] = \sum_{n=0}^{2N-1} x[n] \cos \left[\frac{\pi}{N} \left(n + \frac{1}{2} + \frac{N}{2} \right) \left(k + \frac{1}{2} \right) \right], \quad k = 0, \dots, N-1. \quad (3)$$

The inverse (IMDCT) produces $2N$ samples from the N coefficients. Because the forward transform discards half its inputs’ worth of degrees of freedom, a single IMDCT cannot recover its input; the missing information is supplied by the *overlapping* neighbor block. Each output block overlaps its successor by N samples, and the two contributions are added. This is ***time-domain aliasing cancellation*** (TDAC): the IMDCT of one block contains a time-reversed alias of the input, and the overlapping block’s alias is equal and opposite, so the sum cancels it exactly.

5.1.1 The window and perfect reconstruction

TDAC cancels only if the analysis and synthesis windows satisfy the Princen-Bradley conditions. For a window $w[n]$ of length $2N$, perfect reconstruction requires

$$w[2N - 1 - n] = w[n] \quad (\text{symmetry}), \text{ and} \quad (4)$$

$$w[n]^2 + w[n + N]^2 = 1 \quad (\text{power complementarity}). \quad (5)$$

CELT uses the *Vorbis window*, a power-complementary window built from a sine of a sine:

$$w[n] = \sin\left[\frac{\pi}{2} \sin^2\left(\frac{\pi}{2L} \left(n + \frac{1}{2}\right)\right)\right], \quad n = 0, \dots, L - 1, \quad (6)$$

where L is the overlap length. CELT does not window the full block: only the first and last L samples are tapered (with $L = 120$ at 48 kHz), and the interior is passed through unwindowed. This **low-overlap** design trades a little frequency selectivity for substantially reduced algorithmic delay, which is what lets Opus reach 5 ms latency in CELT mode.

TDAC Reconstruction in the IMDCT

The synthesis overlap-add is the kernel that makes the whole transform invertible. After the IMDCT produces the time-domain block, the overlap region is mixed with the tail held over from the previous frame, applying the window to both edges so that Equation (5) cancels the aliasing:

```
// Mirror both edges for TDAC, mixing with the previous frame's
// tail.
for i in 0..overlap / 2 {
    let (a, b) = (i, overlap - 1 - i);
    let (x1, x2) = (out[b], out[a]);
    let (w1, w2) = (window[i], window[overlap - 1 - i]);
    out[a] = w2 * x2 - w1 * x1;
    out[b] = w1 * x2 + w2 * x1;
}
```

The round trip (forward MDCT then inverse with overlap-add) is verified by a perfect-reconstruction test: a windowed block transformed and inverted returns the input to floating-point precision, with zero delay and unit gain.

5.1.2 Short blocks for transients

A long MDCT gives fine frequency resolution but coarse time resolution, which smears a sharp transient (a drum hit, a consonant) backward in time as **pre-echo**: quantization noise spread across the whole block becomes audible before the attack that produced it. CELT's defense is to switch, for transient frames, from one long MDCT to a series of B short MDCTs (with $B = 2, 4, 8$), each covering a fraction of the frame. The encoder signals this choice with a transient flag (Section 5.7); the short transforms' coefficients are then interleaved so the band structure still applies. Time resolution improves by a factor of B at the cost of frequency resolution, exactly the uncertainty trade-off, applied adaptively per frame.

5.1.3 Computing the MDCT through an FFT

Evaluated directly, Equation (3) costs $O(N^2)$. In practice the MDCT factors into three stages: a complex *pre-rotation* that folds the $2N$ real inputs into $N/4$ complex values, an $N/4$ -point complex FFT, and a complex *post-rotation* that unpacks the result. The FFT dominates the cost and is isolated behind a seam in the implementation: the default build routes it through an external planned-FFT backend, while a dependency-free build falls back to a direct $O(N^2)$ evaluation. The numerical and

performance consequences of this seam are treated in Section 10; here it suffices that the transform is an $O(N \log N)$ operation in the production configuration.

5.2 The Energy Envelope

CELT divides the MDCT spectrum into 21 ***bands*** whose widths follow the ear’s critical bands: narrow at low frequencies where the ear resolves pitch finely, wide at high frequencies where it does not [18]. For each band, CELT codes two things separately: the band’s *energy* (the L^2 norm of its coefficients) and its *shape* (the unit-norm direction). This separation is the “Constrained-Energy” in the codec’s name.

Why Code Energy Separately

The audibility of a frequency band is governed largely by how much energy it contains, not by the precise arrangement of coefficients within it. By coding the energy envelope explicitly and protecting it with its own bits, CELT guarantees that the reconstructed signal has the right spectral balance even when the shape is coded coarsely. A codec that quantized raw coefficients would instead let energy drift as bits run short, producing audible spectral tilt and “birdie” artifacts. The cost of the guarantee is the overhead of the envelope itself, which CELT minimizes with aggressive prediction.

Energy is coded in the $\log_2$ domain in three passes of decreasing significance:

1. **Coarse energy** quantizes each band’s log-energy to roughly 6 dB resolution, predicted from neighbors and coded with a Laplace model.
2. **Fine energy** refines each band with a number of raw bits set by the allocation, halving the error per bit.
3. **Final bits** distribute any leftover budget, one bit at a time, to the bands that the allocation flagged as highest priority.

5.2.1 Coarse energy and two-dimensional prediction

The coarse energy of a band is highly predictable from two neighbors: the same band in the previous frame (time prediction) and the adjacent band in the same frame (frequency prediction). CELT codes only the prediction residual, with a Laplace probability model whose width depends on the frame size. The update applies a prediction coefficient α in time and accumulates a running frequency prediction with a leakage factor β :

$$E_i \leftarrow \alpha E_i^{\text{prev}} + p_i + q_i, \quad p_{i+1} = p_i + q_i(1 - \beta), \quad (7)$$

where q_i is the decoded residual for band i and p_i the frequency predictor carried across bands. Intra frames (no time prediction) set $\alpha = 0$ and use a larger β .

```

let (coef, beta) = if intra { (0.0, BETA_INTRA) }
                    else    { (PRED_COEF[lm], BETA_COEF[lm]) };
let mut prev = [0.0f32; 2];
for i in start..end {
    for (c, prev_c) in prev.iter_mut().enumerate().take(channels) {
        let qi = if budget - tell >= 15 {
            ec_laplace_decode(dec, /* model for band i, frame size lm
            */)
        } else { /* budget-starved fallback code */ };
        let old = state.old_ebands[c][i].max(-9.0);
        state.old_ebands[c][i] = coef * old + *prev_c + qi as f32;
        *prev_c += qi as f32 - beta * qi as f32;
    }
}

```

The Laplace coder (§4.3.2.1) is a specialization of the range coder for a two-sided geometric distribution: small residuals near zero are cheap, large ones progressively more expensive, matching the statistics of inter-frame energy deltas. When the remaining budget falls below the threshold to afford a Laplace symbol, the coder drops to a fixed fallback code so the decode never stalls.

5.3 Pyramid Vector Quantization

Once a band’s energy is known, its shape is a unit vector in N dimensions (for a band of N coefficients). CELT quantizes that direction with **pyramid vector quantization** (PVQ) [3], a structured vector quantizer with a property that makes it ideal here: every codeword has exactly the same norm, so quantizing the shape never disturbs the separately coded energy.

The Pyramid Codebook

The PVQ codebook of dimension N and *pulse count* K is the set of all integer vectors $\mathbf{y} \in \mathbb{Z}^N$ whose absolute components sum to K :

$$S(N, K) = \left\{ \mathbf{y} \in \mathbb{Z}^N : \sum_{i=0}^{N-1} |y_i| = K \right\}. \quad (8)$$

Geometrically these are the lattice points on the surface of an N -dimensional L^1 ball, the “pyramid.” A band’s shape is coded by finding the codeword $\mathbf{y}$ closest in direction to the unit shape, then transmitting which codeword it is. On decode, $\mathbf{y}$ is read back and normalized to unit norm, $\hat{\mathbf{x}} = \mathbf{y}/\|\mathbf{y}\|$, and scaled by the band’s energy. The pulse count K controls the resolution: more pulses mean a denser codebook and a finer shape, and K is exactly what the bit allocation (Section 5.5) decides per band.

5.3.1 The pulse search

Finding the best codeword is a search over $S(N, K)$, which would be enormous to enumerate, so CELT uses a greedy algorithm that adds pulses one at a time, each time placing a pulse in the position that most improves the correlation between the candidate $\mathbf{y}$ and the target shape $\mathbf{x}$. After each pulse, the quantity maximized is

$$j^* = \arg \max_j \frac{(\mathbf{x} \cdot \mathbf{y} + x_j)^2}{\|\mathbf{y}\|^2 + 2y_j + 1}, \quad (9)$$

the squared cosine similarity that would result from adding a pulse at position j . The algorithm starts from a projection of $\mathbf{x}$ onto the pyramid to place most of the pulses cheaply, then refines pulse by pulse. The inner loop of Equation (9) is the single hottest computation in CELT encoding and is the subject of a dedicated SIMD kernel (Section 10.3).

5.4 CWS: Enumerating the Codebook

The PVQ search yields a codeword $\mathbf{y}$; the coder must transmit *which* of the $V(N, K) = |S(N, K)|$ codewords it is, as a uniform integer in $[0, V(N, K))$. This is the **combinatorial** step: a bijection between pulse vectors and indices, known in the Opus sources as CWS (coding with replacement and signs, §4.3.4.2).

Counting Pulse Vectors

Let $V(N, K)$ be the number of length- N integer vectors with K signed pulses, and let $U(N, K)$ count those whose first nonzero entry is positive. These satisfy the recurrences

$$U(N, K) = U(N - 1, K) + U(N, K - 1) + U(N - 1, K - 1), \quad (10)$$

$$V(N, K) = U(N, K) + U(N, K + 1), \quad (11)$$

with $U(0, 0) = 1$. The three terms of Equation (10) correspond to the first position holding zero, holding a pulse with more to come, or holding a pulse that completes the budget. The index of a particular $\mathbf{y}$ is built by walking the positions and, at each, adding the count of all codewords that sort before the chosen one, the standard combinatorial-number-system ranking.

Computing U from Equation (10) on every band decode would cost $O(NK)$. The implementation instead precomputes the U rows into a flat table, so ranking and unranking a codeword become $O(N)$ table lookups. The table has a little over a thousand entries (1272 u32s) and exploits the symmetry $U(n, k) = U(k, n)$ to stay compact. It is built by a `const fn` evaluated at compile time, so the table is baked into the binary with no runtime initialization and no allocator dependency (load-bearing for the `no_std` decoder core, Section 10):

```
// Row m's data starts at column m; PVQ_U_ROW[15] is the total length.
const fn compute_pvq_u_data() -> [u32; PVQ_U_DATA_LEN] {
    let mut uu = [[0u32; 177]; 15]; // U(a, b) for a <= 14, b <= 176
    uu[0][0] = 1;
    let mut a = 1;
    while a < 15 {
        let mut b = 1;
        while b < 177 {
            uu[a][b] = uu[a - 1][b].wrapping_add(uu[a][b - 1]).
                wrapping_add(uu[a - 1][b - 1]);
            b += 1;
        }
        a += 1;
    }
    let mut d = [0u32; PVQ_U_DATA_LEN];
    // ... pack row m of 'uu' into d[PVQ_U_ROW[m]...], starting at
    // column m ...
    d
}

static PVQ_U_DATA: [u32; PVQ_U_DATA_LEN] = compute_pvq_u_data();
```

The decoder reads the uniform index from the range coder, unranks it to recover $\mathbf{y}$ through the table, and normalizes. Because the index is uniform over exactly $V(N, K)$ values, no bits are wasted: the shape costs precisely $\log_2 V(N, K)$ bits, which is what the allocator budgeted for.

5.5 Bit Allocation

The energy envelope and the PVQ shapes both consume bits, and the number of pulses K available to each band is whatever the allocation leaves for it. The allocation is the arbiter that converts a frame's

total bit budget into a per-band pulse count. Its defining constraint is subtle and important.

The Allocation Must Be Implicit

The decoder needs to know each band’s pulse count *before* it can decode that band’s shape, but the pulse counts are never transmitted: there are no bits to spare for them. Instead, encoder and decoder run the *same* deterministic allocation procedure over the same inputs (the total budget, the frame size, static tables, and a few coded parameters), and arrive at identical per-band allocations. This is why the bit budget is tracked to 1/8-bit precision through `tell_frac` (Section 3.6): the allocation arithmetic must produce bit-identical results on both sides, or the decoder will read a band with the wrong pulse count and desynchronize from the range coder. The allocation is, in effect, a pseudo-random function of the budget that both parties compute independently.

The procedure (§4.3.3) works in units of 1/8 bit (the constant `BITRES = 3`). It begins from a family of static *allocation vectors*, tabulated target bit counts per band for a range of quality levels, and interpolates between two adjacent vectors to match the available budget. The interpolation fraction is found by bisection: at each step it tentatively allocates at the midpoint, sums the cost over all bands (clamped to per-band caps, with bands below a floor skipped), and moves the bisection bound according to whether the total overran the budget.

```
// Bisect the interpolation fraction in 1/64 steps to fit 'total'.
let (mut lo, mut hi) = (0i32, 1 << ALLOC_STEPS);
for _ in 0..ALLOC_STEPS {
    let mid = (lo + hi) >> 1;
    let mut psum = 0i32;
    let mut done = false;
    for j in (start..end).rev() {
        let tmp = bits1[j] + ((mid * bits2[j]) >> ALLOC_STEPS);
        if tmp >= thresh[j] || done { done = true; psum += tmp.min(cap[
            j]); }
        else if tmp >= alloc_floor { psum += alloc_floor; }
    }
    if psum > total { hi = mid; } else { lo = mid; }
}
```

Once the per-band bit targets are fixed, each band’s allocation is split between *fine-energy* bits (raw bits that refine the envelope) and *shape* bits (the PVQ codeword). The conversion between a bit budget and a pulse count K goes through a precomputed *pulse cache*: `bits2pulses` binary-searches the cache for the largest K whose codeword fits the budget, and `pulses2bits` reports the exact cost $\lceil \log_2 V(N, K) \rceil$ of a given K . Surplus bits left after the split, the *balance*, are carried forward to subsequent bands so nothing is wasted. The allocation also decides, from the budget, the boundary band for intensity stereo and whether to use dual stereo (Section 5.6), and it may explicitly *skip* high bands when the budget cannot support them, coding the skip decisions as run-length flags.

5.6 Band Coding: Splits, Folding, and Stereo

With energy and allocation settled, the band loop codes each band’s shape. A band of N coefficients is rarely quantized as one PVQ vector; instead it is recursively halved, with the split coded by an angle, until the pieces are small enough to quantize directly. Several refinements layer on top: spreading to decorrelate noisy bands, folding to fill empty ones, anti-collapse to repair transient artifacts, and stereo coupling.

5.6.1 Recursive splitting by angle

When a band is split in two, the energy is divided between the halves by an angle θ : the lower half receives $\cos \theta$ of the amplitude and the upper $\sin \theta$. Quantizing θ (rather than two independent energies) keeps the total band energy exactly fixed, preserving the energy constraint through the recursion. The number of quantization levels for θ depends on the band’s bit allocation, and the angle is coded with one of three probability models chosen by context (§4.3.4.1): a *uniform* PDF for ordinary time splits, a *triangular* PDF that biases toward an even split for mono bands, and a *step* PDF for the first stereo split. The recursion terminates when a sub-band is small enough, at which point it is coded by a single PVQ vector (Section 5.3).

5.6.2 Spreading rotation

Concentrating K pulses in N positions produces a sparse, spiky vector, which in a band that should sound noise-like is audible as tonal “birdies.” CELT counters this by applying a ***spreading rotation***: a sequence of Givens rotations that smear the pulses’ energy across the band without changing its norm. The amount of spreading (none, light, normal, or aggressive) is chosen by the encoder’s tonality analysis (Section 5.7) and signaled once per frame.

5.6.3 Folding and noise fill

When the allocation gives a band zero pulses, leaving it empty would create silent spectral holes. Instead CELT fills the band by ***folding***: copying already-decoded coefficients from lower frequencies into the empty band, then renormalizing to the band’s coded energy. When no suitable lower content exists, the band is filled with pseudo-random noise from a small linear congruential generator. The generator and the fold share the same role, supplying a plausible shape at the correct energy for bands too cheap to code explicitly.

```
// Linear congruential generator used for noise fill (Numerical Recipes
    constants).
fn celt_lcg_rand(seed: u32) -> u32 {
    seed.wrapping_mul(1_664_525).wrapping_add(1_013_904_223)
}
```

5.6.4 Collapse masks and anti-collapse

In a transient frame coded as B short blocks (Section 5.1), a band’s pulses may all land in a single short block, leaving the others at zero, a ***collapse***. A collapsed block is silent for part of the frame, which can sound like a dropout. CELT tracks a ***collapse mask***, one bit per short block per band, recording which blocks received energy. The ***anti-collapse*** step then injects a controlled amount of noise into the collapsed blocks, scaled to the minimum of the neighboring bands’ energies, so every short block carries at least a floor of energy. The presence of anti-collapse is controlled by a single coded bit per transient frame.

5.6.5 Stereo coupling

For stereo input, CELT codes each band as a *mid* (sum) and *side* (difference) pair rather than left and right, because the mid/side representation concentrates energy in the mid channel and codes the stereo image efficiently. Three sub-modes apply, chosen per band by the allocation:

- **Mid/side with angle:** a stereo angle θ apportions energy between mid and side, coded like the mono split above.

- **Intensity stereo:** above a boundary band set by the budget, only the mid is coded and the side is reconstructed by copying the mid, discarding stereo detail where it is least audible (at high frequencies, low bitrates).
- **Dual stereo:** the two channels are coded fully independently, used when the channels are sufficiently decorrelated to justify the bits.

A special case handles bands of $N = 2$, where a single angle suffices to code the stereo relationship directly.

5.6.6 Time-frequency reshaping

Within a frame, individual bands can opt for finer time resolution (at the expense of frequency resolution) or vice versa, independently of the global short-block decision. This is implemented with in-place Haar and Hadamard transforms that redistribute a band’s coefficients between time and frequency before quantization. A per-frame `tf_select` flag and per-band flags encode the choice, which the encoder optimizes by the analysis below.

5.7 The Encoder’s Analysis

Everything to this point is shared by encoder and decoder; the decoder simply reads the parameters and follows. The encoder must *decide* those parameters, and those decisions, not the bitstream format, are where encoders differ in quality. CELT’s encoder runs several analyses per frame.

Transient detection. The encoder decides whether to use short blocks by running the input through a high-pass filter and then forward and backward energy followers; a sharp rise in the high-passed energy relative to its smoothed envelope marks a transient. The high-pass is a second-order section, and its output drives the masking metric:

```
// High-pass filter: (1 - 2z^-1 + z^-2) / (1 - z^-1 + 0.5 z^-2).
let (mut mem0, mut mem1) = (0.0f32, 0.0f32);
for (t, &x) in tmp.iter_mut().zip(input.iter()) {
    let y = mem0 + x;
    let mem00 = mem0;
    mem0 = mem0 - x + 0.5 * mem1;
    mem1 = x - mem00;
    *t = y;
}
```

Time-frequency analysis. For the bands, the encoder chooses time-versus- frequency resolution by an L^1 -metric minimization solved with a small Viterbi search (`tf_analysis` and `tf_select`), trading the cost of changing resolution against the coding gain it yields band by band.

Spreading decision. A tonality measure over the spectrum selects the spreading amount (Section 5.6): tonal frames get little or no spreading, noisy frames get aggressive spreading.

Dynamic allocation and trim. Two analyses shape the allocation before it runs. *Dynamic allocation* (`dynalloc_analysis`) computes per-band importance boosts from a masking model, steering extra bits to perceptually important bands. The *allocation trim* (`alloc_trim_analysis`) sets a single global parameter that tilts the budget toward low or high frequencies based on spectral tilt and transientness.

The pitch pre-filter. Before the MDCT, the encoder optionally applies a comb *pre-filter* that whitens the signal’s harmonic structure, the exact inverse of the comb post-filter the decoder applies after synthesis. Whitening the harmonics concentrates energy and improves coding efficiency on strongly tonal material. The pitch period and gain that parameterize the filter come from a pitch analysis (downsampling, decimated cross-correlation, and octave-error rejection) that mirrors the technique used for packet-loss concealment (Section 7.4).

6 SILK: Linear-Predictive Coding

SILK (§4.2) is the speech half of Opus. Where CELT works in the frequency domain on arbitrary signals, SILK works in the time domain on a *model* of how speech is produced, and codes only what the model fails to predict. This model-based approach is what lets SILK code intelligible speech at a few kilobits per second, far below what a general transform coder achieves. SILK runs at one of three internal rates (8, 12, or 16 kHz, for narrowband, mediumband, and wideband), in frames of 10, 20, 40, or 60 ms divided into 2 or 4 subframes.

This section develops the model bottom-up: the source-filter idea (Section 6.1), the short-term predictor and how it is estimated and represented (Sections 6.2 and 6.3), the long-term predictor that captures pitch (Section 6.4), the gains and excitation coding (Sections 6.5 and 6.6), the noise-shaping quantizer at the encoder’s heart (Section 6.7), and the supporting machinery of stereo, resampling, and control (Sections 6.8 to 6.10).

6.1 The Source-Filter Model

Speech as Source and Filter

Human speech is produced by an excitation source (the vocal folds, for voiced sounds, or turbulent airflow, for unvoiced ones) passing through a filter (the vocal tract, whose shape determines the formants we hear as vowels). Linear-predictive coding [7, 12] models this directly: the vocal tract becomes an all-pole filter $1/A(z)$, and the excitation becomes a sparse residual signal. The model predicts each sample from a weighted sum of recent samples,

$$\hat{s}[n] = - \sum_{k=1}^p a_k s[n-k], \quad (12)$$

and codes only the residual $e[n] = s[n] - \hat{s}[n]$ that the prediction misses. Because the residual carries far less energy and structure than the original, it is cheap to code. SILK adds a second predictor for the residual’s own periodicity (pitch) and a noise-shaping loop that hides quantization noise under the speech spectrum.

The frame is the unit at which SILK transmits its model; the subframe is the unit at which the model is applied. Each frame carries one set of short-term filter coefficients (interpolated across subframes for 20 ms frames), and per-subframe gains and pitch parameters. The decoder reconstructs each frame by running the excitation through the long-term and short-term synthesis filters, then resampling to the output rate.

6.2 Short-Term Prediction

The short-term predictor of Equation (12) is a p -tap linear filter ($p = 10$ for narrowband and medium-band, $p = 16$ for wideband). It comes in two forms that are exact inverses: the **analysis** filter $A(z)$, which the encoder applies to produce the residual, and the **synthesis** filter $1/A(z)$, which the decoder applies to reconstruct the signal. The implementation runs this in Q12 fixed-point on i16 coefficients and samples (SILK’s convention predicts with a *plus* sign, so the analysis filter subtracts the prediction from the input rather than adding it, unlike Equation (12)’s $A(z)$ sign). The taps are reversed once up front so the per-sample dot product is a bounds-check-free forward window:

```

// MA whitening filter (Q12 taps); the first 'order' outputs are zeroed
pub(crate) fn lpc_analysis_filter(out: &mut [i16], input: &[i16], b: &[i16]) {
    let d = b.len();
    let mut b_rev = [0i16; 16];
    for (j, &tap) in b.iter().enumerate() {
        b_rev[d - 1 - j] = tap;
    }
    for ix in d..input.len() {
        let win = &input[ix - d..ix];
        let mut pred_q12 = 0i32;
        for (&w, &tap) in win.iter().zip(b_rev[..d].iter()) {
            pred_q12 = smlabb(pred_q12, i32::from(w), i32::from(tap));
        }
        let e_q12 = (i32::from(input[ix]) << 12) - pred_q12; // e[n]
                    = x[n] - pred
        out[ix] = rshift_round(e_q12, 12).clamp(i32::from(i16::MIN),
            i32::from(i16::MAX)) as i16;
    }
    out[..d].fill(0);
}

```

The synthesis filter run by the decoder is the same recurrence with the sign and the roles of input/output swapped: it accumulates the prediction from *already-reconstructed* output samples and adds it back to the incoming residual, so past decisions feed forward exactly as they did during encoding.

6.2.1 Estimating the coefficients

The decoder receives the coefficients; the encoder must estimate them from the signal. Two classical methods apply. **Levinson-Durbin** recursion [6] solves the normal equations relating the filter coefficients to the signal's autocorrelation, in $O(p^2)$ time, producing the minimum mean-square prediction error. **Burg's method** [1] instead estimates reflection coefficients directly from the signal by minimizing the sum of forward and backward prediction errors, which behaves better on the short analysis windows speech coding uses. The SILK encoder uses a modified Burg method, and caps the resulting filter's prediction gain to keep it numerically well-conditioned.

Levinson-Durbin Recursion

The recursion builds the order- p predictor from the order- $(p-1)$ predictor by computing a reflection coefficient k_m that drives the next prediction error to its minimum, updating all coefficients in place:

```
pub fn levinson_durbin(autocorr: &[f64], order: usize) -> Vec<f64>
{
    let mut a = vec![0.0; order + 1];
    let mut err = autocorr[0];
    for m in 1..=order {
        let mut k = autocorr[m];
        for i in 1..m { k += a[i] * autocorr[m - i]; }
        k = -k / err; // reflection
        coefficient
        a[m] = k;
        for i in 1..=m / 2 { // symmetric in-place
            update
            let tmp = a[i] + k * a[m - i];
            a[m - i] += k * a[i];
            a[i] = tmp;
        }
        err *= 1.0 - k * k; // error shrinks each
        order
    }
    a
}
```

The factor $1 - k_m^2$ is non-negative exactly when $|k_m| \leq 1$, the condition for filter stability, which is why reflection coefficients are a convenient parameterization.

6.3 Line Spectral Frequencies

The encoder cannot transmit LPC coefficients directly: they are extremely sensitive to quantization, and a small rounding error can push the all-pole synthesis filter outside the unit circle, making it unstable and the decoded signal explode. SILK transmits the filter in a different representation, the *line spectral frequencies* (LSFs), which is built for quantization.

Why Line Spectral Frequencies

The order- p polynomial $A(z)$ is decomposed into a symmetric and an antisymmetric polynomial,

$$P(z) = A(z) + z^{-(p+1)} A(z^{-1}), \quad (13)$$

$$Q(z) = A(z) - z^{-(p+1)} A(z^{-1}), \quad (14)$$

whose roots, the LSFs, all lie *on* the unit circle and *interlace* [4]. This representation has three properties that make it ideal for coding. First, stability is trivially checked and enforced: as long as the LSFs stay ordered, the reconstructed filter is stable. Second, the LSFs are localized in frequency, so each one affects the spectrum mainly near its own frequency, and a quantization error in one LSF stays local rather than corrupting the whole spectrum. Third, they interpolate gracefully, so SILK can transmit one set per frame and interpolate across subframes. SILK uses *normalized* LSFs (NLSFs), scaled so the frequency range maps to $[0, 1)$ and stored as Q15 integers.

6.3.1 The two-stage vector quantizer

NLSFs are coded with a two-stage vector quantizer (§4.2.7.5). The first stage selects one of 32 codebook vectors that approximates the whole NLSF vector, coded with an entropy model. The second stage codes the per-coefficient residual between the true NLSFs and the first-stage approximation, using a predictive scheme (each residual partly predicted from the previous) and an arithmetic-coded index. The residual quantization is weighted by *Laroia weights* [5], which approximate each coefficient's perceptual sensitivity (inversely related to its spacing from its neighbors), so bits are spent where the ear notices. The encoder searches this quantizer with a delayed- decision trellis, keeping several candidate paths alive to find a low-rate, low-distortion combination.

6.3.2 Reconstruction and stability enforcement

The decoder reverses the process: dequantize the residual, add the first-stage vector, enforce a minimum spacing between adjacent NLSFs to guarantee stability, and convert back to LPC coefficients. The conversion (`nlsf2a`) builds the P and Q polynomials from the cosines of the NLSFs and combines them. Even after stabilization, the resulting filter is checked for stability, and if it fails, bandwidth expansion (scaling the coefficients toward zero) is applied until it passes:

```
// After converting NLSFs to LPC, force stability by bandwidth
// expansion.
while lpc_inverse_pred_gain(&a_q12[..d]) == 0 && i <
    MAX_LPC_STABILIZE_ITERATIONS {
    bwexpander_32(&mut a32_qa1[..d], 65536 - (2 << i)); // shrink
    toward zero
    /* re-quantize a_q12 from a32_qa1 and re-test */
    i += 1;
}
```

For 20 ms frames, the decoder interpolates between the previous frame's NLSFs and the current frame's, using a 2-bit interpolation factor, so the filter evolves smoothly across the frame's first and second halves rather than jumping at the midpoint.

6.4 Long-Term Prediction

Short-term prediction removes the spectral envelope, but the residual of voiced speech is still strongly periodic at the pitch period: the vocal folds produce a pulse train, and successive pitch periods resemble one another. SILK removes this redundancy with a second predictor, the *long-term predictor* (LTP), which predicts the current residual from the residual one pitch period earlier.

Pitch Prediction with a Five-Tap Filter

For a voiced subframe with pitch lag T (in samples), the LTP predicts the residual $e[n]$ from a five-tap filter applied around $e[n - T]$:

$$\hat{e}[n] = \sum_{j=-2}^2 b_j e[n - T + j]. \quad (15)$$

The five taps b_j let the predictor track a pitch period that does not fall exactly on a sample boundary and that changes shape slowly from period to period. What remains after long-term prediction, the doubly-predicted residual, is the sparse excitation that SILK actually codes. The pitch lag T and the taps b_j are themselves transmitted: the lag as a per-frame absolute value plus per-subframe offsets from a contour codebook, and the taps as an index into one of three gain codebooks.

The lag is decoded as a base lag plus a per-subframe contour offset, clamped to the valid pitch range for the internal rate:

```
let (min_lag, max_lag) = (PE_MIN_LAG_MS * fs_khz, PE_MAX_LAG_MS *
    fs_khz);
let lag = min_lag + i32::from(lag_index);
for (k, out) in pitch_lags.iter_mut().enumerate().take(nb_subfr) {
    *out = (lag + contour_offset(k)).clamp(min_lag, max_lag); // per-
        subframe lag
}
```

The five-tap gains come from one of three codebooks of 8, 16, or 32 vectors, selected by a per-frame *periodicity index* that trades codebook size against the bits to address it: highly periodic speech justifies the larger, more precise codebook. The encoder chooses the lag with a three-stage pitch search (Section 6.10) and the gains with a rate-distortion-weighted vector quantizer.

6.5 Subframe Gains

Each subframe carries a gain that scales the excitation before it enters the synthesis filters, tracking the signal’s changing loudness. Gains are quantized in the $\log_2$ domain across 64 levels, so the quantization step is a constant ratio rather than a constant difference, which matches the multiplicative way loudness varies. After the first subframe, gains are delta-coded against the previous subframe, with a wider “double step” available for rapid changes. The decoder converts the log-domain index back to a linear Q16 gain through a piecewise-parabolic $\log_2$ -to-linear approximation:

```
for k in 0..nb_subfr {
    if k == 0 && !conditional {
        // The index may not drop more than 16 steps (~21.8 dB).
        *prev_ind = i32::from(ind[k]).max(i32::from(*prev_ind) - 16) as
            i8;
    } else {
        let ind_tmp = i32::from(ind[k]) + MIN_DELTA_GAIN_QUANT; //
            delta-coded
        // Accumulate deltas, with a wider double step above a
            threshold.
        let double_step_threshold =
            2 * MAX_DELTA_GAIN_QUANT - N_LEVELS_QGAIN + i32::from(*
                prev_ind);
        *prev_ind = if ind_tmp > double_step_threshold {
            i32::from(*prev_ind) + ((ind_tmp << 1) -
                double_step_threshold)
        } else {
            i32::from(*prev_ind) + ind_tmp
        } as i8;
    }
    *prev_ind = i32::from(*prev_ind).clamp(0, N_LEVELS_QGAIN - 1) as i8
        ;
    // Scale into the log domain and back to linear Q16.
    gain_q16[k] = log2lin((smulwb(INV_SCALE_Q16, i32::from(*prev_ind))
        + OFFSET).min(3967));
}
```

6.6 The Excitation and the Shell Coder

After both predictors and the gain, what remains to code is the *excitation*: the quantized residual, a mostly-small integer per sample. SILK codes it in 16-sample blocks with a *shell coder* (§4.2.7.8), a

recursive scheme that codes the *distribution* of pulses within a block rather than each sample independently.

Recursive Pulse Splitting

Given the total number of pulses in a 16-sample block, the shell coder splits that total down a binary tree: the 16-sample count divides into two 8-sample counts, each into two 4-sample counts, and so on down to individual samples. At each node only the *split* is coded, that is, how the parent's pulse count divides between its two children, using a probability model conditioned on the parent count. This is efficient because the total pulse count already constrains the splits: a block with two pulses has far fewer ways to distribute them than a block with ten, and the conditional models capture exactly that. Pulse magnitudes beyond what the tree codes directly are sent as additional least-significant-bit planes, and each non-zero sample's sign is coded afterward with a probability that depends on the signal type, the quantization offset, and the block's pulse count.

```
// One node of the shell tree: split a parent count into two child counts.
fn decode_split(dec: &mut RangeDecoder, p: i32, shell_table: &[u8]) ->
    (i32, i32) {
    if p > 0 {
        let off = SHELL_CODE_TABLE_OFFSETS[p as usize] as usize;
        let child1 = dec.decode_icdf(&shell_table[off..], 8) as i32;
        (child1, p - child1) // the other child is the remainder
    } else {
        (0, 0)
    }
}
```

A per-frame *rate level* selects which family of shell probability tables to use, letting the excitation coder adapt its models to the bitrate.

6.7 The Noise-Shaping Quantizer

The noise-shaping quantizer (NSQ) is the central kernel of the SILK encoder, the stage that actually decides the excitation values. A naive quantizer would round the residual to the nearest level independently per sample; the NSQ does something more sophisticated, choosing each excitation sample to shape the spectrum of the resulting quantization *noise* so that it hides beneath the speech.

Shaping Quantization Noise

Quantization injects error, and that error has a spectrum. If the error spectrum is flat (white), it is audible wherever the speech is quiet. The insight of noise shaping is to spectrally color the error so it follows the speech spectrum, concentrating noise at frequencies where the speech is loud enough to mask it [12]. The NSQ achieves this by running the quantization inside a feedback loop: the error from past samples is filtered by a *noise-shaping filter* and fed back into the current prediction, so the quantizer is steered to make errors whose spectrum matches a perceptual target. Because the loop reconstructs exactly the signal the decoder will produce, the encoder always knows the true reconstruction state, which is what allows the per-sample decisions to be optimal rather than open-loop.

For each sample the NSQ forms the short-term and long-term predictions, adds the shaping feedback, quantizes the result by a rate-distortion rule (minimizing $|\text{error}| + \lambda \cdot \text{bits}$, where λ comes from the target coding quality), emits the excitation value, and updates its filter states. The prediction and

feedback dot products are computed in fixed point, bit-identically to the reference, so the encoder’s reconstruction matches the decoder’s to the sample:

```
// The LPC prediction (Q10). ‘coef_rev’ is reversed once per subframe
// so this
// descending-history access becomes a forward windowed dot with no
// bounds
// checks in the per-sample loop.
fn short_prediction(buf: &[i32], base: usize, coef_rev: &[i16]) -> i32
{
    let order = coef_rev.len();
    let w = &buf[base + 1 - order..=base];
    let mut out = (order >> 1) as i32;
    for (&wj, &cj) in w.iter().zip(coef_rev.iter()) {
        out = smlawb(out, wj, i32::from(cj)); // scalar SMLAWB chain
    }
    out
}
```

This one is deliberately scalar rather than routed through the SIMD dot products of Section 10.2: it runs once per sample over at most 16 taps, and a SIMD call’s fixed overhead (the call itself, and folding the lanes back to a scalar) does not amortize over so few taps. The scalar/SIMD line is drawn per call site by measured cost, not by a blanket rule.

6.7.1 Noise-shaping analysis

The filters the NSQ applies are designed once per frame by the *noise-shaping analysis*. It examines the speech and produces four shaping controls: an auto-regressive (AR) filter derived from the LPC envelope and bandwidth-expanded to set the overall noise shape; a spectral *tilt* that emphasizes or de-emphasizes low frequencies; a low-frequency shaper; and, for voiced frames, a *harmonic shaper* that shapes noise around the pitch harmonics. These are driven by the target SNR and the voice-activity analysis so that the noise follows the masking threshold as the signal changes.

6.8 Stereo Prediction

Stereo SILK codes a mid channel (the average of left and right) and a predicted side channel (their difference), because the side is usually well predicted from the mid and so costs few bits. The encoder estimates per-frame prediction weights that reconstruct the side from a three-tap smoothed mid plus a one-tap lagged mid; the weights are quantized, coded, and interpolated over the frame’s first 8 ms to avoid discontinuities. When the side is sufficiently predictable, a *mid-only* flag drops it entirely. The decoder applies the inverse:

```
// Reconstruct the side channel from the mid using interpolated
// predictor weights,
// then convert mid/side back to left/right.
for n in 0..frame_length {
    let sum = i32::from(x1[n + 1]) + i32::from(x2[n + 1]); // L = mid
    // + side
    let diff = i32::from(x1[n + 1]) - i32::from(x2[n + 1]); // R = mid
    // - side
    x1[n + 1] = sum.clamp(i16::MIN as i32, i16::MAX as i32) as i16;
    x2[n + 1] = diff.clamp(i16::MIN as i32, i16::MAX as i32) as i16;
}
```

6.9 Resampling

SILK runs at 8, 12, or 16 kHz internally, but the caller asks for 8, 12, 16, 24, or 48 kHz output, so the decoder ends with a resampling stage (§4.2.8). The method depends on the ratio: equal rates pass through, a factor-of-two change uses a cascade of second-order allpass sections (cheaper and lower-delay than a long FIR for exact $2\times$), and other ratios combine the $2\times$ allpass with a fractional-delay FIR interpolation. The encoder’s front-end runs the inverse, decimating 48 kHz input down to the internal rate; its half-band and 2/3-band decimators are pinned bit-exactly against the reference, since the resampled signal is what the rest of the encoder analyzes.

6.10 Voice Activity, Pitch, and Rate Control

Three encoder analyses decide the parameters the rest of SILK consumes.

Voice activity detection. The VAD splits each frame into four frequency bands with a tree of allpass half-band filters, tracks the noise floor in each band, and maps the per-band signal-to-noise ratios through a sigmoid to a speech-activity probability (a Q8 value). This probability, along with a spectral-tilt measure, drives the noise-shaping analysis, the gain control, and the pitch threshold, so that the codec spends its effort where there is speech.

Pitch analysis. The pitch lag is found by a three-stage search of increasing precision: a coarse cross-correlation at 4 kHz proposes candidate lags, a refinement at 8 kHz searches a small contour codebook around them, and a final search at the internal rate locks in the per-subframe lags. The search also classifies the frame as voiced or unvoiced, which selects whether the long-term predictor is used at all.

Rate control. A target bitrate is translated into a target coding SNR through per-bandwidth tables, with an adjustment for the shorter 10 ms frames. The coding SNR then sets the rate-distortion trade-off λ in the NSQ and the aggressiveness of the noise shaping. When a coded frame would exceed a hard byte budget, the gains are scaled coarser and the frame re-encoded until it fits, which is how SILK honors a bitrate cap and how the hybrid mode keeps room for CELT’s high band (Section 7.2).

6.11 Packet-Loss Concealment in SILK

When a SILK frame is lost, the decoder conceals it by extrapolating the last good frame’s model: it continues the pitch excitation with progressively stronger attenuation, drifts the pitch lag, and fades toward the spectral envelope of the preceding frames, mixing in comfort noise estimated from the recent signal. The attenuation and pitch-drift constants are tuned so a short loss is inaudible and a long one decays gracefully to silence rather than repeating a buzz. This is the SILK-specific counterpart to the general concealment logic in Section 7.4.

7 The Opus Layer

SILK and CELT are complete coders on their own. The Opus layer (§4 for the decoder, §5 for the encoder) is what makes them one codec: it dispatches each frame to the right coder, runs both together in hybrid mode, smooths the transitions when the mode changes between frames, and implements the resilience features (concealment, forward error correction, discontinuous transmission) that operate above either coder. This layer holds the cross-frame state that neither coder can see on its own.

7.1 Mode Dispatch

The TOC byte names the mode; the decoder dispatches accordingly. A frame’s bytes become a single range decoder, and the SILK half (if any) decodes first, followed by the CELT half (if any), both drawing from the same range decoder so the bitstream is one continuous arithmetic-coded message.

After both halves, the internal output is resampled to the API rate and any cross-frame smoothing is applied. The skeleton of the per-frame decode is:

```
let mut dec = RangeDecoder::new(data);
if mode != Mode::CeltOnly {
    // SILK half: low band (SILK-only) or 0..8 kHz (hybrid), at 8/12/16
    kHz.
    self.silk.decode(&mut dec, &ctl, /* first call */, &mut pcm_silk);
    for (p, &s) in pcm.iter_mut().zip(pcm_silk.iter()) {
        *p += f32::from(s) / 32768.0;
    }
}
// ... optional redundancy handling ...
if mode != Mode::SilkOnly {
    // CELT half: full band (CELT-only) or bands 17..end (hybrid).
    let start_band = if mode == Mode::CeltOnly { 0 } else { 17 };
    pcm = self.celt.decode_frame(&mut dec, len, frame_size,
                                self.stream_channels, start_band,
                                celt_end);
}
self.final_range = dec.range_size() ^ redundant_rng;
```

7.2 Hybrid Mode and the Crossover

Hybrid mode is the reason SILK and CELT share a range coder. In a hybrid frame, SILK codes the signal up to roughly 8 kHz and CELT codes everything above, and the two contributions are summed (SILK's after resampling its 16 kHz output to 48 kHz) to reconstruct the full-band signal.

Splitting the Spectrum Between Two Coders

CELT's band structure assigns a fixed range of MDCT bands to each frequency region. In hybrid mode, CELT simply starts at band 17 (about 8 kHz) instead of band 0, leaving the bands below for SILK. The crossover is not a filter but a bookkeeping boundary: SILK is given the low band as its full-band wideband signal, and CELT is told to begin coding at band 17. Because both write into one range coder, the decoder reads the SILK parameters, then the CELT parameters, with no marker between them; the byte where SILK ends and CELT begins is wherever the SILK decode happens to leave the range coder. The highest CELT band depends on the audio bandwidth: super-wideband stops at band 19, fullband at band 21.

The encoder side must budget the shared bytes so that CELT always has room for its high band. It computes a SILK bitrate from a rate table, caps the SILK low band to a hard byte budget, and lets CELT fill the remainder, all into one range encoder:

```
let celt_floor = (celt_end - 17) * 3 + 3;           // min bytes for the
    high band
let silk_cap = nb_bytes.saturating_sub(celt_floor).max(8);
let mut enc = RangeEncoder::new(nb_bytes);
silk.encode_into(&mut enc, &internal, Some((silk_cap * 8) as i32)); //
    capped
self.celt.encode_hybrid_into(&mut enc, pcm, nb_bytes, celt_end);    //
    fills rest
```

7.3 Mode Transitions and Redundancy

Because the mode can change from one packet to the next, the decoder must bridge discontinuities when, say, a SILK-only packet is followed by a CELT-only packet. The coders carry independent

internal state (filter memories, overlap buffers), and switching cold would click. Opus addresses this with two mechanisms: state resets at the boundary, and an optional *redundancy* frame.

Redundancy Frames at Transitions

A redundancy frame is a short (5 ms) CELT frame embedded in a SILK or hybrid packet, used to fill the gap during a mode switch. When switching from CELT to SILK, the redundant CELT frame covers the start of the SILK packet, which has no valid CELT history; when switching from SILK to CELT, it covers the end. The decoder cross-fades between the redundant frame and the primary decode over a 2.5 ms overlap using a squared Vorbis window, so the energy sums to unity across the splice and no discontinuity is heard. The presence and direction of redundancy are coded with a few bits at the boundary of the SILK and CELT data.

The crossfade is the same power-complementary mix used throughout Opus, applied sample by sample over the overlap:

```
fn smooth_fade(in1: &[f32], in2: &[f32], out: &mut [f32],
               overlap: usize, channels: usize, inc: usize) {
    for c in 0..channels {
        for i in 0..overlap {
            let w = WINDOW120[i * inc] * WINDOW120[i * inc]; //
                squared window
            out[i * channels + c] =
                w * in2[i * channels + c] + (1.0 - w) * in1[i *
                    channels + c];
        }
    }
}
```

7.4 Packet-Loss Concealment

When a packet does not arrive, the decoder must produce a plausible frame of audio rather than a gap. This is *packet-loss concealment* (PLC). Opus conceals in whichever mode was last active: the CELT concealer extrapolates the last frame's pitch period with decaying gain, and the SILK concealer continues its excitation model (Section 6.11). A loss is signaled to the decoder by handing it an empty or single-byte packet, which it routes to the concealer for the expected frame duration:

```
if data.len() <= 1 {
    return Ok(self.decode_lost(self.last_frame_size / self.ds));
}
```

7.5 In-Band Forward Error Correction

Concealment hides a loss but cannot recover the lost content. Opus can do better for speech through *in-band forward error correction* (FEC): a SILK or hybrid packet may carry a second, low-bitrate copy of the *previous* frame, called *LBRR* (low-bitrate redundancy). If a packet is lost but the next one arrives, the decoder extracts the LBRR copy of the lost frame from the new packet and decodes it instead of concealing, recovering a degraded but real version of the audio. FEC is engaged through a separate decode entry point that, on loss, prefers the LBRR data when present and conceals only when it is absent. The cost is a modest bitrate increase on the carrying packets, which the encoder budgets only when packet loss is expected.

7.6 Discontinuous Transmission

During silence there is nothing to code, and continuing to send full frames wastes bandwidth. *Discontinuous transmission* (DTX) lets the encoder stop sending audio frames during inactivity, emitting only occasional one-byte TOC packets that the decoder conceals as comfort noise. The decision is driven by an activity detector that tracks the background noise floor, so DTX engages during genuine pauses even when there is audible ambient noise, not only during digital silence:

```
fn frame_active(&mut self, pcm: &[f32]) -> bool {
    let ms = pcm.iter().map(|&v| v * v).sum::<f32>() / pcm.len() as f32
    ;
    // Track the minimum energy with a slow upward leak (the noise
    floor).
    self.dtx_noise_floor = if ms < self.dtx_noise_floor { ms }
                           else { self.dtx_noise_floor * 1.0003 };
    ms > (self.dtx_noise_floor * ACTIVE_RATIO).max(ABS_FLOOR)
}
```

After a configured period of inactivity, the encoder drops to TOC-only packets, cutting the silence bitrate to a fraction of a kilobit per second, and resumes full coding the moment activity returns.

7.7 The Encoder's Mode Selection

The encoder's top level chooses a mode per frame from the frame size and target bitrate. Short frames (2.5 and 5 ms) can only be CELT; long frames (40 and 60 ms) can only be SILK; the 10 and 20 ms sizes admit all three, and the choice falls to SILK for low-bitrate speech bandwidths, hybrid for the higher bandwidths at moderate rates, and CELT otherwise. Ahead of every mode, the input passes through a 3 Hz one-pole high-pass that removes DC and sub-audible rumble before coding, with its state carried across frames so the filter does not restart each frame.

8 Ogg Encapsulation

The packet layer (Section 4) deliberately stores no length, assuming a container provides framing. For stored files, that container is Ogg (RFC 3533 [9]), with an Opus-specific mapping (RFC 7845 [14]) that defines the headers and timing. This section covers how Opus packets are written to and read from an .opus file: the page structure, the checksum, packet reassembly, the two mandatory headers, and the granule timing that synchronizes audio to a clock.

8.1 Pages and Lacing

Ogg's transport unit is the *page*, not the packet. A page is a self-contained, checksummed run of bytes that begins with the four-byte capture pattern `OggS` and carries a header (flags, a granule position, the stream's serial number, a page sequence number, and a CRC) followed by a *segment table* and the segment data.

Lacing: Packets into Segments

A page does not store packet lengths directly. Instead its body is divided into *segments* of up to 255 bytes, and the segment table lists each segment's length as one byte. A packet is encoded as a sequence of 255-byte segments followed by one final segment of length less than 255, so a segment value below 255 marks the end of a packet and a value of exactly 255 means the packet continues into the next segment. A packet whose length is an exact multiple of 255 therefore ends with a zero-length segment, which is how the reader distinguishes “the packet happened to fill the last segment” from “the packet continues.” Because a page holds at most 255 segments, a packet too long for one page spills into the next, with a continuation flag marking the follow-on page.

8.2 The Ogg CRC

Every page carries a 32-bit CRC over its entire contents (with the CRC field itself zeroed during computation), so a reader can detect corruption and a mistaken sync to a false OggS pattern. The polynomial is the standard 0x04C11DB7, but, unusually, Ogg uses the *non-reflected* variant with zero initial value and no final inversion, which differs from the more common reflected CRC-32 of zlib or Ethernet.

```
const POLY: u32 = 0x04C1_1DB7;          // non-reflected, zero init, no
final XOR

pub(crate) const fn update(mut crc: u32, data: &[u8]) -> u32 {
    let mut i = 0;
    while i < data.len() {
        crc = (crc << 8) ^ TABLE[((crc >> 24) as u8 ^ data[i]) as usize];
        i += 1;
    }
    crc
}
```

When a page fails its CRC, the reader does not give up: it scans forward for the next OggS capture pattern and resynchronizes, so a single corrupt page costs only its own audio, not the rest of the stream.

8.3 Packet Reassembly

Reading Opus packets from an Ogg file is the inverse of lacing: walk the pages of the chosen logical stream (matched by serial number), concatenate segments until a terminator (a segment under 255 bytes) completes a packet, and carry a partial packet across a page boundary when the page ends mid-packet. The reader tracks the page sequence number to detect gaps, and discards a buffered partial packet if continuity breaks, so a lost page cannot fuse two unrelated packets:

```
// A page ends mid-packet (last lacing == 255) -> buffer the partial;
otherwise
// the accumulated bytes complete a packet. A sequence gap drops the
partial.
if !consecutive || page.continued != self.have_partial {
    self.partial.clear();
    self.have_partial = false;
}
```

A defensive cap bounds the size of a reassembled packet, so hostile input cannot drive unbounded allocation; a packet that exceeds it is dropped rather than buffered.

8.4 OpusHead and OpusTags

An Ogg Opus stream begins with two mandatory header packets, each alone on its own page. The first, *OpusHead*, is the identification header: it carries the version, the channel count, the *pre-skip* (Section 8.5), the original input sample rate (informational only, since decoding is always at 48 kHz internally), an output gain, and the channel mapping family that determines whether the stream is mono/stereo or multichannel surround (Section 9). The second, *OpusTags*, is the comment header: a vendor string and a list of TAG=value metadata entries, in the same format as Vorbis comments.

```
pub struct OpusHead {
    pub version: u8,                // 1
    pub channel_count: u8,
    pub pre_skip: u16,              // 48 kHz samples to discard at
        startup
    pub input_sample_rate: u32,     // informational
    pub output_gain_q8: i16,        // Q7.8 dB, applied by the
        player
    pub channel_mapping: ChannelMapping,
}
```

8.5 Granule Positions and Timing

Ogg pages carry a *granule position*, a per-codec timestamp. For Opus it counts samples at 48 kHz, the same reference clock used everywhere in the codec (Section 2.4). The granule position of a page is the total number of decoded samples available after decoding every packet that *completes* on that page, which lets a player seek and compute durations precisely.

Pre-Skip and End-Trim

Two timing offsets account for the codec's own latency. The encoder's filters (the SILK resampler, the CELT overlap) introduce a fixed delay, so the first decoded samples are startup transient rather than real audio. *Pre-skip*, stored in OpusHead, tells the player how many leading 48 kHz samples to discard so the output aligns with the original input. At the other end, the final page's granule position can fall short of the samples the last packet actually produces, signaling *end-trim*: the player discards the trailing samples beyond the granule position. Together these let an Opus file represent an audio signal of *any* sample count exactly, even though the codec only produces audio in fixed-size frames. The reader resolves per-packet positions by walking backward from a page's granule position, subtracting each packet's duration (read from its TOC), so every packet gets a precise timestamp.

The implementation provides both directions: a reader that parses headers, reassembles packets, and resolves their granule positions, and a writer that emits the two header pages, then audio pages with correctly computed granule positions and a final end-of-stream page. The round trip is interoperable with the reference tools: files written by the implementation decode in ffmpeg/libopus at the correct duration, and files those tools produce decode here.

9 Multistream and Surround

Opus codes mono or stereo. Surround content (5.1, 7.1, ambisonics) is handled not by extending the codec but by running several Opus streams in parallel and multiplexing them into one packet, the *multistream* format (RFC 7845 [14] §5.1.1). This keeps the core codec simple: it never needs to know about more than two channels.

9.1 Streams and Coupling

A multistream packet contains N Opus streams. The first M of them are ***coupled*** (stereo, carrying two channels each) and the remaining $N - M$ are ***uncoupled*** (mono), so the streams together carry $2M + (N - M) = M + N$ decoded channels. A channel mapping table then routes those decoded channels to the output layout.

Coupled and Uncoupled Streams

Pairing channels into a coupled (stereo) stream lets Opus exploit the correlation between them, exactly the mid/side coding of Sections 5.6 and 6.8, which is more efficient than coding two channels independently. Channels that are largely independent (a surround channel and the front left, say) are better coded as separate mono streams. An encoder partitions the output channels into coupled pairs and mono singletons, records that partition as the stream count N and coupled count M , and stores a ***channel mapping***: one entry per output channel giving which decoded channel feeds it, or a sentinel meaning silence. A 5.1 layout, for instance, might use three coupled streams and no mono streams, with the mapping assigning the six decoded channels to front, center/LFE, and surround positions.

9.2 Demultiplexing

The streams are concatenated in one packet, which raises the framing problem the core format avoided: the decoder must find where each stream's bytes end. Here the ***self-delimiting*** framing from Section 4.1 earns its place. Every stream except the last is written self-delimited, announcing its own length, so the demultiplexer can parse them in order; the last stream is parsed normally and consumes the remainder. Each stream is then decoded by its own Opus decoder, and the results are scattered into the output according to the mapping:

```
for s in 0..streams {
  let packet = if s != streams - 1 {
    let (packet, used) = Packet::parse_self_delimited(rest)?; //
    knows its length
    rest = &rest[used..];
    packet
  } else {
    Packet::parse(rest)? //
    takes the rest
  };
  stream_pcm.push(self.decoders[s].decode_parsed(&packet));
}
// Route decoded channels to output channels (255 == silence).
for (ch, &m) in self.mapping.iter().enumerate() {
  if m == 255 { continue; }
  /* copy decoded channel m into output channel ch */
}
```

All streams in a packet must decode to the same number of samples, which the demultiplexer enforces; a mismatch is rejected as malformed. The channel mapping family in OpusHead (Section 8.4) selects the convention: family 0 is plain mono/stereo, and the higher families define the surround channel orders and the stream/coupling counts.

10 Computational Optimizations

The preceding sections describe what Opus computes. This section describes how the implementation makes those computations fast enough for real-time use, decoding at hundreds of times realtime on one

core, and encoding faster than the reference at its default complexity. The optimizations fall into two groups: those that are free because they change only encoder *decisions*, and those that preserve the bitstream exactly. Understanding which is which requires returning to the asymmetry introduced in Section 2.5.

10.1 The Licence to Approximate

Why the Encoder May Approximate but the Decoder May Not

The Opus bitstream is defined by the sequence of range-coder symbols (Section 3), not by the floating-point values an encoder computed to choose them. Any *valid* choice the decoder can follow produces a legal stream. This splits the codec’s arithmetic into two regimes with different rules. The decoder, and every encoder computation that feeds a coded symbol whose exact value the decoder recomputes (the bit allocation, the energy prediction, the NSQ reconstruction), must be *bit-exact*: a single diverging bit desynchronizes the range coder. But an encoder computation that only *informs a decision*, such as which pitch lag looks best or which pulse vector maximizes correlation, may use any approximation that lands on a valid answer, because the decoder never recomputes it; it just follows the symbols that result. Every SIMD kernel below lives in the second regime, which is why fast, approximate arithmetic is provably harmless there.

This is also why the implementation can keep `unsafe` confined to a handful of SIMD kernels. The crate denies `unsafe` by default; each kernel opts in explicitly with a documented safety justification, and each sits on the encoder’s decision path where the range coder, not the kernel’s exact output, defines the bitstream. Every round-trip and conformance test passes whether the SIMD path or the scalar fallback runs.

10.2 SIMD Dot Products

The dominant cost of encoding is correlation: the pitch analyses in both SILK and CELT, the LPC whitening filters, and the NSQ prediction all reduce to dot products over long buffers. The implementation provides vectorized dot products with runtime CPU-feature detection, dispatching to an AVX2+FMA kernel when available, an SSE2 kernel as the x86-64 baseline, and a scalar fallback elsewhere:

```
pub(crate) fn dot(x: &[f32], y: &[f32]) -> f32 {
    #[cfg(target_arch = "x86_64")]
    // SAFETY: both slices expose >= x.len() contiguous f32s; the
    // kernels read
    // only those lanes (vector loads masked to width, with a scalar
    // tail).
    unsafe { if has_avx2() { dot_avx2(x, y) } else { dot_sse2(x, y) } }
    #[cfg(not(target_arch = "x86_64"))]
    dot_scalar(x, y)
}
```

The feature detection is cached in an atomic so the check runs once, not per call. One subtlety bridges the two regimes of Section 10.1: SILK’s pitch decisions are pinned against the reference, so the helper that feeds them accumulates in an `f64` even though it operates on `f32` data, keeping the SIMD result within the reference’s tolerance. The CELT pitch correlations and the encoder’s other analyses feed pure decisions and tolerate FMA’s fused rounding freely.

10.3 The SIMD PVQ Search

The pulse search of Equation (9) is the hottest loop in CELT encoding: for each of K pulses, it scans all N positions for the one maximizing a correlation ratio. The implementation vectorizes this with SSE2 and AVX2 kernels, and it is the clearest illustration of the licence to approximate.

Two Tricks: Reciprocal Square Roots and Sentinel Lanes

The correlation ratio of Equation (9) needs a division by $\|\mathbf{y}\|$, computed across lanes with the hardware *reciprocal square root* instruction, a fast 12-bit approximation rather than an exact divide. This can occasionally pick a different position than the scalar search would, but the result is always a *valid* pulse vector, so it round-trips exactly through the decoder; only the encoder's choice shifts, never the bitstream's legality. The second trick handles the vector tail: the work buffers are padded to a whole number of SIMD lanes, and the padding lanes are filled with sentinel values (a very negative numerator and a very large denominator) that can never win the argmax. This lets every iteration process a full vector with no remainder branch and no bounds check, while guaranteeing a padding lane is never selected.

The work buffers are thread-local and reused across bands and frames, so the search allocates nothing in steady state. This pattern, reusing scratch buffers rather than allocating per call, recurs across the hot paths: the PVQ buffers, the MDCT scratch, and the FFT plans are all cached per thread.

10.4 The MDCT Pre-Rotation

The MDCT's pre-rotation (Section 5.1) folds the real input into complex values by a per-element complex multiply against precomputed twiddle factors. Storing the twiddles in *folded* interleaved form (real and imaginary parts adjacent) lets an SSE2 kernel deinterleave four complex elements, multiply, and reinterleave in a few instructions, replacing a scalar complex-multiply loop. Because the MDCT runs on the synthesis path as well, this kernel's correctness is guarded by the same TDAC perfect-reconstruction test that validates the transform itself (Section 5.1).

10.5 The FFT Backend Seam

The largest single decode speedup comes not from a kernel but from a seam. The MDCT's inner FFT (Section 5.1) is isolated behind a function boundary with two implementations selected at build time.

Two FFT Backends Behind One Interface

The dependency-free build evaluates the FFT directly, an $O(N^2)$ computation that keeps the crate free of dependencies for embedded and minimal-footprint targets. The default build instead routes the FFT through an external planned-FFT backend, an $O(N \log N)$ implementation, yielding roughly a tenfold decode speedup on the conformance vectors. The seam matters because the two backends produce subtly different floating-point results, and the codec's conformance criterion is PCM *quality*, not bit-exact PCM (Section 11): both backends pass, differing only in rounding far below the audibility threshold. Isolating the FFT this way lets the implementation be fast by default and dependency-free by choice, without forking the codec.

10.6 Table-Driven Enumeration and Buffer Reuse

Two further optimizations are algorithmic rather than architectural. The CWRS enumeration (Section 5.4) replaces the $O(NK)$ recurrence with $O(N)$ lookups into a table computed once at compile time (a `const fn` baked into the binary, not a runtime lazy-init), which is significant because band decoding calls it on every band of every frame. And throughout the hot paths, per-call heap allocation is eliminated by reusing thread-local scratch buffers: the band-encode work vectors, the MDCT complex

scratch, and the cached FFT plans all persist across calls, so steady-state coding does no allocation at all. Combined with the `no_std`-compatible core, this keeps the codec’s memory behavior predictable enough for real-time and embedded use.

11 Numerical Considerations and Conformance

Opus is specified with unusual precision about arithmetic, because much of it must be reproduced exactly across implementations while the rest is deliberately left free. This final section explains where bit-exactness is required, how fixed-point and floating-point arithmetic divide the codec, and how an implementation proves itself correct against the official conformance suite.

11.1 Fixed-Point and Floating-Point

Q-Format Fixed-Point Arithmetic

A Qn value stores a real number x as the integer $\lfloor x \cdot 2^n \rfloor$, so the fraction lives in the low n bits. Addition is ordinary integer addition; multiplying a Qa value by a Qb value gives a $Q(a + b)$ product that is shifted right to renormalize. Fixed-point arithmetic is exactly reproducible on every machine, with no dependence on a floating-point unit’s rounding modes, which is why the normative parts of the codec use it. The cost is constant vigilance about scaling and overflow: the implementation matches the reference’s wrapping integer semantics precisely, using explicit wrapping operations where the reference relies on machine wraparound, so that intermediate sums agree to the bit.

The two coders make different choices. SILK is specified in fixed point, and the implementation follows suit: its NLSF conversion, gain dequantization, LTP filters, and NSQ run in Q-format integers pinned bit-exactly against the compiled reference, so SILK reconstructs sample-identical PCM. CELT’s reference exists in both fixed-point and floating-point forms, and RFC 6716 explicitly permits a floating-point decoder; the implementation uses float for CELT, which makes its PCM output differ from a fixed-point decoder by rounding far below audibility, while its range-coder arithmetic remains exact.

11.2 What Must Be Exact

The dividing line follows the range coder. Anything that produces or consumes a coded symbol must be bit-exact, because the symbols are the bitstream and the range coder desynchronizes on any divergence. Anything downstream of the last symbol, the final PCM samples after the inverse transform and resampling, need only be perceptually correct.

Important Note

The single invariant that captures “bit-exact” is the final-range oracle of Section 3.7: after decoding a packet, the range coder’s `rng` must equal the reference encoder’s, exactly. This holds even for the floating-point CELT decoder, because the float arithmetic only affects the PCM *after* the last symbol is decoded; every symbol along the way is decoded with identical integer range-coder arithmetic. An implementation can therefore use float for speed in the transform and still prove, packet by packet, that it decoded exactly the same bitstream the reference encoded.

11.3 The Conformance Criterion

RFC 6716 ships a suite of test vectors and a comparison tool, `opus_compare`, that scores a decoder’s PCM output against the reference on a perceptually weighted metric. An implementation conforms if it passes this quality bar on every vector; bit-exact PCM is *not* required, precisely because the standard permits floating-point decoders whose output differs in rounding.

The implementation discussed here passes the official criterion: all twelve vectors score in the 99.2–100% quality range, well above the passing bar, with per-packet final ranges bit-exact across the whole suite. The combination is strong evidence of correctness: the final-range equality proves the entropy decode is exact symbol-for-symbol, and the quality score proves the signal reconstruction is perceptually faithful. For the CELT-only vectors, the synthesized PCM additionally scores 83–104 dB SNR against the reference float decode, far beyond what the quality metric demands.

Two Checks, Two Purposes

The two conformance measures answer different questions. The **final-range oracle** answers “did I decode the same bitstream?”, a binary, unforgiving check on the entropy coder and everything that feeds it; a single wrong symbol fails it. The **opus_compare quality score** answers “does the audio sound right?”, a graded check on the signal path that tolerates the rounding differences a floating-point decoder legitimately introduces. Passing both means an implementation is exact where the standard demands exactness and faithful where it demands fidelity, which is the strongest practical guarantee a decoder can offer.

12 Python API Reference

12.1 Introduction

`ruopus` ships a Python extension module built with `PyO3` and `NumPy` interop (`rust-numpy`). It mirrors the Rust public surface described in Section 7 almost one-to-one: a stateful `OpusEncoder` and `OpusDecoder`, the packet enums (`Mode`, `Bandwidth`, `FrameSize`, `Application`, `Signal`), an exception hierarchy rooted at `OpusError`, and a `ruopus.lowlevel` submodule exposing the SILK and CELT layers directly.

Zero-Copy PCM

PCM crosses the Rust/Python boundary as `NumPy float32` (or `int16`) arrays shaped `(frames, channels)`, with no extra copy in either direction: decode output is moved out of Rust’s `Vec` directly into the array’s buffer, and encode input is borrowed from the array via the buffer protocol. Every codec call releases the GIL, so decode throughput scales across threads exactly as the pure-Rust benchmarks in Section 10 do.

Fully type-hinted stubs (`.pyi`, generated from the compiled extension by `PyO3`’s `experimental-inspect` and checked with `pyright`) and one set of docstrings (sourced from the Rust `///` comments and served to both IDE tooling and the Sphinx-built HTML guide at <https://jmg049.github.io/ruopus/>) ship in every wheel.

12.2 Installation

```
pip install ruopus
```

Pre-built wheels cover CPython 3.9 through 3.13 on Linux, macOS, and Windows; no Rust toolchain is required. `NumPy 1.22+` is the only runtime dependency. Building from source (`maturin build --features python`) needs a `Q1.85+` Rust toolchain, matching the crate’s MSRV.

12.3 Quickstart: Encode and Decode

The encoder and decoder are both stateful classes; construct one of each per stream and feed them consecutive frames so cross-frame state (MDCT overlap, mode hysteresis, concealment history) stays continuous.

```
import numpy as np
import ruopus

# Stereo encoder at 64 kbps (default: fullband, complexity 10, auto
  mode)
enc = ruopus.OpusEncoder(2, bitrate=64_000)
dec = ruopus.OpusDecoder(2)

# 20 ms stereo frame at 48 kHz -> 960 samples per channel
frame = np.zeros((960, 2), dtype=np.float32)

packet = enc.encode_auto(frame)          # bytes: the encoded Opus packet
pcm     = dec.decode_packet(packet)      # (960, 2) float32 in [-1, 1]
```

Encoder input accepts a 1-D interleaved array or a 2-D (**frames**, **channels**) array, both without extra copies. Mono streams use **channels=1**; decoded PCM is then (**frames**, 1). For callers that need integer samples, `OpusDecoder`'s `decode_packet_i16` method returns `int16` PCM instead, scaled and rounded exactly as `opus_demo` does.

Every encoder property (`bitrate`, `complexity`, `dtx`, `bandwidth`, `signal`, `application`, `inband_fec`, `packet_loss_perc`, `vbr`, `force_channels`) is writable at runtime and takes effect on the next frame – there is no need to rebuild the encoder to change quality or bitrate mid-stream.

12.4 Choosing a Codec Mode

Section 7 describes SILK, CELT, and hybrid mode at the bitstream level; the Python encoder exposes one method per mode plus an automatic dispatcher:

```
enc.encode_auto(frame)      # per-frame SILK / hybrid / CELT decision
enc.encode(frame)           # force CELT-only
enc.encode_silk(frame)      # force SILK-only (speech, 10/20/40/60 ms)
enc.encode_hybrid(frame)    # force hybrid (10/20 ms, SWB/FB only)
```

`encode_auto` is the right default for most applications: it runs the encoder’s signal analysis (Section 7.7) every frame and picks a mode from the `signal` hint, the `application` profile, the target bitrate, and cross-frame hysteresis. The `Signal` and `Application` enums bias that decision explicitly:

```
# Phone-quality voice
enc_voice = ruopus.OpusEncoder(
    1, bitrate=8_000,
    application=ruopus.Application.Voip,
    signal=ruopus.Signal.Voice,
)

# High-quality music
enc_music = ruopus.OpusEncoder(
    2, bitrate=128_000,
    application=ruopus.Application.Audio,
    signal=ruopus.Signal.Music,
)

# Low-latency game voice (CELT only, never SILK)
enc_ld = ruopus.OpusEncoder(
    1, bitrate=32_000,
    application=ruopus.Application.RestrictedLowDelay,
)
```

After encoding, `ruopus.Packet(packet).toc` reports which mode, bandwidth, and frame size the encoder actually chose (Section 12.9).

12.5 Frame Sizes

Opus’s six frame durations (Table 4) are always expressed in samples *per channel* at 48 kHz, regardless of which mode encodes them. `FrameSize` carries both directions of that conversion:

```
fs = ruopus.FrameSize.Ms20
print(fs.tenth_ms, fs.samples_per_channel_48k)    # 200, 960
```

20 ms is the only duration every mode accepts and the standard default. Passing a duration one of `encode`, `encode_silk`, or `encode_hybrid` does not support raises `EncodeError`. `enc.lookahead` reports the encoder’s algorithmic delay in samples (120 for the default fullband CELT configuration, i.e. the MDCT overlap of Section 5.1) – skip that many leading output samples when aligning a round-trip for testing.

Duration	Samples @ 48 kHz	Modes
2.5 ms	120	CELT only
5 ms	240	CELT only
10 ms	480	SILK, hybrid, CELT
20 ms	960	SILK, hybrid, CELT
40 ms	1920	SILK only
60 ms	2880	SILK only

Table 4: Valid frame durations and the modes that accept each.

12.6 Ogg Opus Files

Section 8 describes the Ogg container at the bitstream level; `encode_ogg_opus/decode_ogg_opus` handle the entire round-trip – header, pages, pre-skip, output gain – in one call each:

```
import numpy as np
import ruopus

sr = 48_000
t = np.linspace(0, 3, sr * 3, dtype=np.float32)
pcm = np.column_stack([
    np.sin(2 * np.pi * 440 * t), # left: A4
    np.sin(2 * np.pi * 880 * t), # right: A5
])

ogg = ruopus.encode_ogg_opus(pcm, channels=2, bitrate=128_000)
with open("output.opus", "wb") as f:
    f.write(ogg)

with open("output.opus", "rb") as f:
    decoded_pcm, head = ruopus.decode_ogg_opus(f.read())

print(f"{head.channel_count}-ch, {decoded_pcm.shape[0]}/{sr:.2f}s")
```

Decoding applies the `OpusHead` pre-skip (Section 8.5), trims trailing granule padding, and applies the header’s Q7.8 output gain, so the returned PCM is ready to play or save as-is. `OpusHead` exposes every identification-header field (`version`, `channel_count`, `pre_skip`, `input_sample_rate`, `output_gain_q8`, and the channel-mapping fields for non-family-0 streams) and round-trips back to bytes via `to_bytes()`. Only channel-mapping family 0 (mono/stereo) is supported by `decode_ogg_opus`; surround files need the multistream decoder (Section 12.8).

12.7 Packet Loss and Forward Error Correction

Sections 7.4 and 7.5 describe concealment and in-band FEC at the bitstream level. The decoder exposes both directly:

```
import ruopus

dec = ruopus.OpusDecoder(2)
FRAME = 960 # 20 ms at 48 kHz

for pkt in stream:
    if pkt is None:
        pcm = dec.decode_lost(frame_size=FRAME) # concealment
    else:
        pcm = dec.decode_packet(pkt)

# Recover a lost packet from in-band FEC carried by its successor
recovered = dec.decode_fec(next_pkt, frame_size=FRAME)
```

`decode_lost` extrapolates the last pitch period after a CELT/hybrid packet; a lost frame following SILK fades to silence (SILK’s own PLC is not yet ported – Section 6.11). `decode_fec` recovers a lost frame from the low-bitrate redundant (LBRR) copy the *next* packet carries, falling back to plain concealment when no usable FEC data is present. Enable FEC generation on the encoder with two properties:

```
enc = ruopus.OpusEncoder(
    1, bitrate=24_000,
    application=ruopus.Application.Voip,
    signal=ruopus.Signal.Voice,
    inband_fec=True,
    packet_loss_perc=10, # hint: expect 10% loss
)
```

`packet_loss_perc` (clamped 0–100) biases the encoder toward loss-robust coding – more redundancy at the cost of raw efficiency – and is only effective on SILK/hybrid packets, so it has no effect under `Application.RestrictedLowDelay`.

12.8 Multistream and Surround Decoding

Section 9 describes multistream demultiplexing at the bitstream level. `MultistreamDecoder` decodes `streams` elementary Opus streams per packet (the first `coupled` as stereo, the rest mono) and routes the resulting channels to an output layout via `mapping`, where the sentinel 255 produces a silent channel:

```
import ruopus

# 5.1 surround: 4 elementary streams, 2 stereo-coupled
dec = ruopus.MultistreamDecoder(
    streams=4,
    coupled=2,
    mapping=[0, 4, 1, 2, 3, 5], # L R C LFE Ls Rs
)
pcm = dec.decode_packet(raw_packet) # (frames, 6) float32
```

Each packet’s elementary streams are self-delimited (Section 4.1) except the last, which is standard-framed; `decode_packet` handles that transparently. Mono/stereo Ogg files (channel-mapping family 0) never use multistream framing, so `decode_ogg_opus` handles them without this class.

12.9 Packet Introspection

`Toc` and `Packet` parse Opus framing (Section 4) without decoding audio – useful for logging, monitoring, or building test vectors:

```
import ruopus

pkt = ruopus.Packet(raw_bytes)
print(f"mode={pkt.toc.mode}, bandwidth={pkt.toc.bandwidth}, frames={len(pkt)}")

for frame in pkt.frames:      # each a 'bytes' object; may be empty (
    DTX)
    ...
first_frame = pkt[0]         # Packet is a read-only sequence of
    frames
```

`Toc` exposes the raw byte plus its three decoded fields (`mode`, `bandwidth`, `frame_size`, `channels`, `stereo`, `config`, `frame_count_code`) and is hashable, so TOC distributions across a stream can be tallied with a `Counter`. `Packet.parse` validates the R1–R7 framing rules (Section 4.4) and raises `PacketError` on a malformed packet; `Packet.parse_self_delimited` parses the self-delimiting form used inside multistream payloads and returns the bytes consumed alongside the packet. After a round-trip, comparing `enc.final_range` against `dec.final_range` (Section 3.7) is the same bit-exactness oracle used by the Rust conformance suite.

12.10 The Low-Level API

`ruopus.lowlevel` exposes the SILK and CELT layers of Sections 5 and 6 directly, below the Opus packet format, plus the LPC analysis/synthesis pipeline underlying SILK. These are research and custom-codec building blocks; ordinary use should prefer `OpusEncoder`/`OpusDecoder`.

```
import numpy as np
import ruopus.lowlevel as ll

# The CELT layer alone: raw frame bodies, no TOC, no Opus framing
enc = ll.CeltEncoder(channels=1, complexity=10, bitrate=64_000)
frame = np.zeros(960, dtype=np.float32)
body = enc.encode(frame)          # raw CELT frame body

# SILK operates at its own internal rate (8/12/16 kHz), not 48 kHz,
# and on int16 PCM
silk_enc = ll.SilkEncoder(fs_khz=16, nb_subfr=4, bitrate=25_000)
payload = silk_enc.encode(np.zeros(320, dtype=np.int16)) # 20 ms @ 16
    kHz

# The LPC analysis pipeline that drives SILK's short-term prediction
signal = np.random.randn(320).astype(np.float32)
coeffs = ll.lpc_analysis(signal, order=16)          # Levinson-
    Durbin
residual = ll.lpc_residual_stateful(signal, coeffs)
lag = ll.estimate_pitch(signal, fs_khz=16)          # long-term
    prediction
```

`SilkStereoEncoder` extends `SilkEncoder` with mid-side coding (Section 6.8); `DecControl` carries the SILK decoder's per-frame control parameters. `CeltDecoder` does not expose a full `decode` method, since CELT decoding needs the encoder's live range-coder state (Section 3.7), which crosses no API

boundary – real CELT packets decode through `OpusDecoder` instead; the low-level decoder covers concealment (`decode_lost`) independently.

12.11 Complete API Reference

Name	Purpose
<code>OpusEncoder</code> / <code>OpusDecoder</code>	Top-level Opus packet codec (Section 12.3)
<code>MultistreamDecoder</code>	Surround/multistream decode (Section 12.8)
<code>Packet</code> / <code>Toc</code>	Packet and TOC introspection (Section 12.9)
<code>OpusHead</code>	Ogg Opus identification header (Section 12.6)
<code>encode_ogg_opus</code> / <code>decode_ogg_opus</code>	Whole-file Ogg Opus helpers (Section 12.6)
<code>Mode</code> , <code>Bandwidth</code> , <code>FrameSize</code>	Packet-level enums (Section 4)
<code>Signal</code> , <code>Application</code>	Encoder mode/latency hints (Section 12.4)
<code>OpusError</code>	Base exception for every codec failure
<code>PacketError</code> , <code>EncodeError</code> , <code>OggError</code>	Specific exception subclasses
<code>ruopus.lowlevel</code>	SILK/CELT/LPC primitives (Section 12.10)
<code>ruopus.version()</code>	The installed <code>ruopus</code> version string

Table 5: The top-level `ruopus` Python surface. Every class and function carries a full docstring and type stub; run `help(ruopus.X)` or consult <https://jmg049.github.io/ruopus/> for exhaustive parameter-level documentation.

Important Note

This reference is deliberately a map, not the territory: exhaustive parameter documentation lives in the generated `.pyi` stubs (so IDEs show it inline) and the Sphinx guide. Anywhere this section and the installed package disagree, the package is correct – file an issue against the manual.

References

- [1] John Parker Burg. “Maximum Entropy Spectral Analysis”. In: *Ph.D. dissertation, Stanford University* (1975).
- [2] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. 2nd. Wiley-Interscience, 2006.
- [3] Thomas R. Fischer. “A Pyramid Vector Quantizer”. In: *IEEE Transactions on Information Theory* 32.4 (1986), pp. 568–583. DOI: [10.1109/TIT.1986.1057198](https://doi.org/10.1109/TIT.1986.1057198).
- [4] Fumitada Itakura. “Line Spectrum Representation of Linear Predictive Coefficients of Speech Signals”. In: *The Journal of the Acoustical Society of America* 57.S1 (1975), S35. DOI: [10.1121/1.1995189](https://doi.org/10.1121/1.1995189).
- [5] Rajiv Laroia, Nam Phamdo, and Nariman Farvardin. “Robust and Efficient Quantization of Speech LSP Parameters Using Structured Vector Quantizers”. In: *Proc. IEEE ICASSP* (1991), pp. 641–644. DOI: [10.1109/ICASSP.1991.150421](https://doi.org/10.1109/ICASSP.1991.150421).
- [6] Norman Levinson. “The Wiener RMS Error Criterion in Filter Design and Prediction”. In: *Journal of Mathematics and Physics* 25.1–4 (1947), pp. 261–278. DOI: [10.1002/sapm1946251261](https://doi.org/10.1002/sapm1946251261).
- [7] John Makhoul. *Linear Prediction: A Tutorial Review*. Vol. 63. 4. 1975, pp. 561–580. DOI: [10.1109/PROC.1975.9792](https://doi.org/10.1109/PROC.1975.9792).
- [8] G. Nigel N. Martin. “Range Encoding: An Algorithm for Removing Redundancy from a Digitised Message”. In: *Video and Data Recording Conference, Southampton* (1979).
- [9] Silvia Pfeiffer. *The Ogg Encapsulation Format Version 0*. Request for Comments RFC 3533. Internet Engineering Task Force (IETF), May 2003. DOI: [10.17487/RFC3533](https://doi.org/10.17487/RFC3533). URL: <https://www.rfc-editor.org/rfc/rfc3533>.
- [10] John P. Princen and Alan B. Bradley. “Analysis/Synthesis Filter Bank Design Based on Time Domain Aliasing Cancellation”. In: *IEEE Transactions on Acoustics, Speech, and Signal Processing* 34.5 (1986), pp. 1153–1161. DOI: [10.1109/TASSP.1986.1164954](https://doi.org/10.1109/TASSP.1986.1164954).
- [11] John P. Princen, A. W. Johnson, and Alan B. Bradley. “Subband/Transform Coding Using Filter Bank Designs Based on Time Domain Aliasing Cancellation”. In: *Proc. IEEE ICASSP* (1987), pp. 2161–2164. DOI: [10.1109/ICASSP.1987.1169405](https://doi.org/10.1109/ICASSP.1987.1169405).
- [12] Lawrence R. Rabiner and Ronald W. Schafer. *Digital Processing of Speech Signals*. Prentice-Hall, 1978.
- [13] Claude E. Shannon. “A Mathematical Theory of Communication”. In: *The Bell System Technical Journal* 27.3 (1948), pp. 379–423. DOI: [10.1002/j.1538-7305.1948.tb01338.x](https://doi.org/10.1002/j.1538-7305.1948.tb01338.x).
- [14] Timothy B. Terriberry, Ron Lee, and Ralph Giles. *Ogg Encapsulation for the Opus Audio Codec*. Request for Comments RFC 7845. Internet Engineering Task Force (IETF), Apr. 2016. DOI: [10.17487/RFC7845](https://doi.org/10.17487/RFC7845). URL: <https://www.rfc-editor.org/rfc/rfc7845>.
- [15] Jean-Marc Valin and Timothy B. Terriberry. *Updates to the Opus Audio Codec*. Request for Comments RFC 8251. Internet Engineering Task Force (IETF), Oct. 2017. DOI: [10.17487/RFC8251](https://doi.org/10.17487/RFC8251). URL: <https://www.rfc-editor.org/rfc/rfc8251>.
- [16] Jean-Marc Valin, Koen Vos, and Timothy B. Terriberry. *Definition of the Opus Audio Codec*. Request for Comments RFC 6716. Internet Engineering Task Force (IETF), Sept. 2012. DOI: [10.17487/RFC6716](https://doi.org/10.17487/RFC6716). URL: <https://www.rfc-editor.org/rfc/rfc6716>.
- [17] Ian H. Witten, Radford M. Neal, and John G. Cleary. “Arithmetic Coding for Data Compression”. In: *Communications of the ACM* 30.6 (1987), pp. 520–540. DOI: [10.1145/214762.214771](https://doi.org/10.1145/214762.214771).
- [18] Eberhard Zwicker and Hugo Fastl. *Psychoacoustics: Facts and Models*. 2nd. Springer, 1999.